

Low-Latency Bootstrapping for CKKS using Roots of Unity

Jean-Sébastien Coron and Robin Köstler

University of Luxembourg

Abstract. We introduce Sparse Roots of Unity (SPRU) bootstrapping, a new bootstrapping algorithm for the CKKS homomorphic encryption scheme for approximate arithmetic. The original CKKS bootstrapping method relies on homomorphically evaluating a polynomial that approximates modular reduction modulo q . In contrast, SPRU bootstrapping directly embeds the additive group modulo q into the complex roots of unity, which can be evaluated natively in the CKKS scheme. This approach significantly reduces the multiplicative depth required for bootstrapping, enabling the use of a smaller ring dimension and improving efficiency. In practice, using the OpenFHE C++ library, SPRU bootstrapping achieves up to a $5\times$ reduction in latency when applied to ciphertexts with a small number of slots.

1 Introduction

Fully Homomorphic Encryption. Homomorphic encryption (HE) enables to perform operations on encrypted data without knowing the decryption key. *Fully homomorphic encryption* (FHE) extends this capability, enabling the evaluation of arbitrary circuits on encrypted data. FHE has a wide range of applications, including secure cloud computing, multi-party computation, and secure machine learning.

Since Gentry’s invention of FHE in 2009 [Gen09], numerous homomorphic encryption schemes have been developed, leading to significant improvements in FHE performance. Gentry’s key innovation was the introduction of *bootstrapping*, a ciphertext refreshing procedure based on homomorphically evaluating the decryption circuit. Building on this breakthrough, schemes such as BGV [BGV11] and BFV [Bra12,FV12] further enhanced FHE by basing their security on the hardness of the ring learning with errors (RLWE) problem [SSTX09,LPR10]. These schemes can achieve high computational throughput by utilizing Single Instruction, Multiple Data (SIMD) operations [GHS12a,SV14].

The FHEW scheme, introduced in [DM15], reduced the time for bootstrapping a single-bit encryption to under one second, building on the approach from [AP14] and the matrix-based scheme of [GSW13]. This was further improved in [CGGI16] with the torus-FHE (TFHE) scheme, achieving a bootstrapping time of less than 0.1 second.

The CKKS scheme. In [CKKS17], the authors presented a construction for homomorphic encryption supporting approximate arithmetic, based on the RLWE problem. The core idea involves adding noise to the plaintext to ensure security, with this noise being treated as part of the error inherent in approximate computations. Given a ciphertext ct , for a secret key sk , the decryption equation $\langle ct, sk \rangle \pmod{q}$ outputs an approximation $m + e$ of the original message m , for a small error e . The authors introduced a rescaling procedure to control the magnitude of plaintexts, enabling the construction of a leveled homomorphic encryption scheme, where the ciphertext modulus grows linearly with the depth of the evaluated circuit. Through rescaling, the scheme can emulate fixed-point addition and multiplication on encrypted messages. Additionally, they described a packing method that allows the encryption of up to $N/2$ complex numbers in a single ciphertext to perform SIMD operations.

CKKS bootstrapping. In [CHK⁺18a], the authors introduced a novel ciphertext refreshing procedure for CKKS, extending the leveled homomorphic encryption scheme to fully homomorphic encryption based on Gentry’s bootstrapping technique. Specifically, the initial leveled scheme from [CKKS17] can only evaluate circuits of a fixed depth, as each homomorphic multiplication reduces the ciphertext modulus until it becomes too small to support further computation.

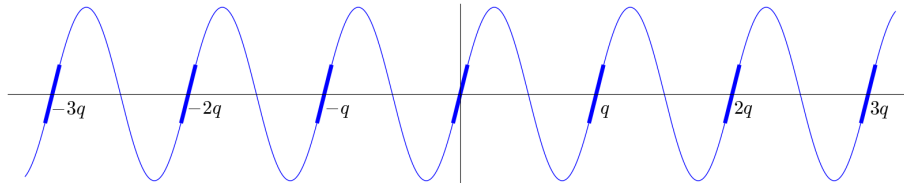


Fig. 1. Approximating modular reduction via a scaled sine function.

As illustrated in Figure 1, CKKS bootstrapping relies on approximating the modular reduction modulo q using a scaled sine function. Since the sine function is 2π -periodic, the decryption function can be expressed as:

$$[\langle \text{ct}, \text{sk} \rangle]_q \approx \frac{q}{2\pi} \cdot \sin \left(\frac{2\pi}{q} \cdot \langle \text{ct}, \text{sk} \rangle \right).$$

which is periodic modulo q and closely approximates $\langle \text{ct}, \text{sk} \rangle$ near 0. Given that $m \ll q$, the polynomial approximation only has to be good for small values modulo q . The bootstrapping procedure homomorphically evaluates this polynomial approximation, producing a refreshed ciphertext that encrypts the same message but under a larger modulus, thereby enabling further computation. The error resulting from bootstrapping is kept sufficiently small to preserve plaintext precision.

The authors also demonstrated how to refresh packed ciphertexts containing n slots with $\mathcal{O}(n)$ complexity. In particular, they use a linear transformation that moves the polynomial coefficients into the plaintext slots via the *coefficient-to-slot* (CoeffToSlot) procedure, and reverses the latter using the *slot-to-coefficient* (SlotToCoeff) operation. The complexity of these operations was further reduced to $\mathcal{O}(\log n)$ in [CHH18, CCS19]. We recall the original CKKS bootstrapping procedure in more details in Appendix C, first for a single slot, and then for multiple slots, using the CoeffToSlot and SlotToCoeff operations.

The main drawback of CKKS bootstrapping is its relatively high runtime, primarily due to its substantial multiplicative depth, which necessitates a large modulus. Reducing the multiplicative depth would free up more levels for useful computations after bootstrapping, thereby improving overall throughput. Additionally, it would allow for the selection of a smaller ring dimension, leading to reduced latency.

Sparse Roots of Unity (SPRU) bootstrapping. Our main contribution is SPRU bootstrapping, a new bootstrapping algorithm for CKKS that achieves reduced multiplicative depth. The key idea is to embed the additive group $\mathbb{Z}_q$ into the complex roots of unity via the homomorphism:

$$f : \mathbb{Z}_q \longrightarrow \mathbb{C}, \quad x \longmapsto \exp(2i\pi \cdot x/q).$$

This embedding enables modular reduction to be evaluated natively within the CKKS scheme, avoiding expensive polynomial approximations and enabling more efficient ciphertext refreshing, especially for a small number of slots.

Considering for simplicity an LWE ciphertext $\mathbf{c}$, we can compute the decryption equation $m = \langle \mathbf{s}, \mathbf{c} \rangle = \sum_{k=0}^{N-1} s_k c_k \pmod{q}$ over this multiplicative group, using $s_k \in \{0, 1\}$ as a selector:

$$\exp(2i\pi \cdot \langle \mathbf{s}, \mathbf{c} \rangle / q) = \prod_{k=0}^{N-1} \exp(2i\pi \cdot c_k s_k / q) = \prod_{k=0}^{N-1} (1 + (\exp(2i\pi \cdot c_k / q) - 1) \cdot s_k).$$

Moreover, considering a message $m \in \mathbb{Z}$ such that $m = [\langle \mathbf{c}, \mathbf{s} \rangle]_q \ll q$, we can use the approximation $\exp(x) \simeq 1 + x$ for small x , which yields:

$$1 + m \cdot \frac{2i\pi}{q} \simeq \prod_{k=0}^{N-1} (1 + (\exp(2i\pi \cdot c_k / q) - 1) \cdot s_k) \quad (1)$$

We claim that this provides a new bootstrapping equation for CKKS, as it can be homomorphically evaluated to provide a refreshed ciphertext. Specifically, it suffices to encrypt each s_k , and following (1), homomorphically multiply it by $\exp(2i\pi \cdot c_k / q) - 1$. The full product is then computed homomorphically using CKKS multiplications. Eventually, by extracting the imaginary part and applying appropriate scaling, we obtain a new ciphertext encrypting the same $m \in \mathbb{Z}$, but under a larger modulus $q \cdot p$ than the original modulus q , enabling further computation.

This bootstrapping equation differs significantly from the original CKKS bootstrapping algorithm, as we avoid using a polynomial approximation of the sine function. Instead, the modular reduction modulo q is achieved by directly embedding the ciphertext coefficients $c_k \in \mathbb{Z}$ into the complex circle group, where computations are performed using CKKS' homomorphic operations. Notably, when bootstrapping multiple slots, the *coefficient-to-slot* procedure from the CKKS bootstrapping is dispensable, as the ciphertext is already embedded within the complex slots. However, the *slot-to-coefficient* (STC) operation is still needed to revert to the coefficient representation.

Our approach shares conceptual similarities with the blind rotation algorithms used in FHEW/TFHE bootstrapping [DM15, CGGI16], where the additive group $\mathbb{Z}_q$ is embedded into the set Y^i of roots of unity of $X^N + 1$, for $N = k \cdot q$ and $Y = X^{2k}$. In both cases, the computation involves a chain of products with factors selected homomorphically over the encrypted coefficients of the secret key. However, we stress that the underlying mathematics of FHEW/TFHE and CKKS are fundamentally different. FHEW/TFHE bootstrapping is optimized for low-latency processing of single-slot messages with limited precision (typically Boolean to 5-bit integers). In contrast, CKKS supports the encryption and bootstrapping of up to $N/2$ high-precision complex values in a single ciphertext, albeit at the cost of significantly higher latency.

The goal of this paper is therefore to explore this new bootstrapping approach, prove a corresponding bootstrapping theorem, and introduce several optimizations that make it practical. Finally, we present concrete parameter sets and benchmark our results based on the OpenFHE C++ library [BAB⁺22], comparing the performance of our bootstrapping with the original CKKS bootstrapping.

First optimization: packing the secret-key bits. We introduce a first optimization for the homomorphic evaluation of our bootstrapping equation (1), where the N secret-key bits s_i are packed into the $N/2$ slots of CKKS ciphertexts. As a consequence, the bootstrapping key size decreases from N to only two CKKS ciphertexts. The product of the slots is then computed using a product operator (corresponding to the standard field norm, see Sec. 3.5), most notably

reducing the number of ciphertext-ciphertext multiplications from $\mathcal{O}(N)$ to $\mathcal{O}(\log N)$. When homomorphically evaluating (1), we further take advantage of CKKS natively supporting complex numbers in the slots.

Second optimization: a sparse block secret-key. As a second optimization, we assume that the secret key is structured into h blocks, each of size $B = N/h$, with exactly one non-zero bit per block. This technique was previously explored in the context of TFHE bootstrapping [LMSS23]. Leveraging this structure, we can rewrite the bootstrapping equation (1) using partial sums over each block instead of products. For a packed ciphertext, these sums are efficiently computed using the trace operator, while the remaining products are still handled by the product operator. Since most homomorphic multiplications are replaced by additions, this approach reduces the multiplicative depth from $\log_2 N + 1$ to $\log_2 h + 1$. Additionally, the number of homomorphic multiplications decreases from $\mathcal{O}(\log N)$ to $\mathcal{O}(\log h)$, further improving efficiency. We formally describe our bootstrapping algorithm for a single slot by combining the two previous optimizations in Section 4.4.

Bootstrapping any number of slots. Equation (1) applies to bootstrapping a single slot $m \in \mathbb{Z}$, but it can be naturally extended to handle multiple slots. We first generalize the approach to n slots for $n \leq B/2$, where B is the block size in the secret key. To achieve this, we treat each coefficient of the polynomial message as the decryption of an independent LWE ciphertext. The key bits and the corresponding embeddings of the LWE ciphertext coefficients are then packed into the $N/2$ slots of $2n$ CKKS ciphertexts. As before, the sum is computed in two steps: first, by summing across $2n$ distinct ciphertexts, and then by applying the trace operator to the result. The product is again evaluated using the product operator. Finally, we apply the `SlotToCoeff` operation to move the slots into the polynomial coefficients. Note that, as opposed to the original CKKS bootstrapping, we do not need the `CoeffToSlot` operation. To support ciphertexts with up to $n \leq N/2$ slots, we apply the above bootstrapping procedure sequentially over ciphertexts containing $B/2$ slots.

Asymptotically, our method has a complexity of $\mathcal{O}(n + \log N)$ homomorphic operations, whereas the original CKKS bootstrapping scales as $\mathcal{O}(\log n + \log N)$. Consequently, the original CKKS bootstrapping is more efficient for large n . However, for a small number of slots, our approach is significantly faster in practice due to its lower multiplicative depth. Additionally, our bootstrapping method is highly parallelizable; with n processors, it achieves the same $\mathcal{O}(\log N)$ complexity as the original CKKS bootstrapping.

Open-source implementation based on OpenFHE. We present our open-source implementation of the new bootstrapping algorithm, developed using the OpenFHE C++ library. This enables a direct comparison with the existing implementation of the original CKKS bootstrapping provided in OpenFHE. We recall the parameter set used for the original bootstrapping and introduce a concrete parameter set for our method, both ensuring a 128-bit security level. Our results demonstrate that, in practice, the reduced multiplicative depth of our approach yields up to a $5\times$ speedup when bootstrapping ciphertexts with a small number of slots. The source code is provided in

https://github.com/coron/spru_boot

Efficient implementations of CKKS. Efficient implementations of CKKS use a *residue number system* (RNS), which decomposes the ciphertext modulus into several smaller moduli. This allows operations to be performed on native 64-bit word-sized integers, providing significant computational advantages. The first RNS-CKKS scheme was introduced in [CHK⁺18b], achieving an order of magnitude improvement in efficiency, by adapting the double-CRT representation in the BGV scheme [GHS12b]. Since the RNS technique is implemented in the OpenFHE C++ library, we are able to directly compare our method against the state-of-the-art implementation of the original CKKS bootstrapping available in OpenFHE.

A significant portion of CKKS’ runtime is attributed to its linear transformations, specifically `SlotToCoeff` and `CoeffToSlot`, which perform homomorphic encoding and decoding. These transformations can be executed using matrix multiplication, requiring $\mathcal{O}(n)$ homomorphic operations [CHK⁺18a]. This was later optimized to $\mathcal{O}(\log n)$ by employing a homomorphic Discrete Fourier Transform (DFT) [CHH18, CCS19]. Both techniques are implemented in the OpenFHE C++ library for the original CKKS bootstrapping. Our bootstrapping implementation reuses the existing `SlotToCoeff` implementation without modification.

Another optimization involves GPU-accelerated implementations, as described in [SYL⁺23] and [JKA⁺21]. Additionally, the authors of [CKKS23] proposed a new technique for performing a ciphertext multiplication that consumes fewer bits of the ciphertext modulus. All these optimizations are fully compatible with our new bootstrapping algorithm.

Related work. The CKKS approximation of the modular reduction function, specifically the `EvalMod` algorithm, has been optimized in [KPK⁺23] and [LLK⁺22] to consume fewer bits of the ciphertext modulus. The approximation of the sine function has been improved in [LLL⁺21] and [JM22], improving the bootstrapping precision.

In [KDE⁺21], the authors introduced a new bootstrapping approach for BGV, BFV and CKKS, based on the blind rotation technique used in FHEW/TFHE bootstrapping. For CKKS, their approach enables high-precision bootstrapping with a relatively small ring dimension of $N = 2^{13}$. Since FHEW/TFHE can only bootstrap a single slot, the ciphertext is split according to the number of slots. Then, each part is bootstrapped independently, and the results are recombined into a single ciphertext. However, this approach requires $\mathcal{O}(N)$ homomorphic operations for a single slot, which could render it impractical for CKKS. The authors do not provide timing results for their implementation.

In [BCC⁺22], the authors introduced a method for achieving high-precision bootstrapping by combining multiple low-precision bootstrapping steps. As a result, it is possible to keep the same parameters (in particular, the same modulus size) for bootstrapping while achieving higher precision, albeit with the trade-off of requiring more iterations. Finally, it was recently shown that CKKS can be used in a black-box manner to perform computations on binary data [DMPS24]. This was further improved in [BCKS24] with a CKKS bootstrapping algorithm designed specifically for ciphertexts encoding binary data.

Independent work. Recently, Cheon, Hanrot, Kim, and Stehlé introduced a new bootstrapping algorithm for CKKS at Eurocrypt 2025 [CHKS25]. Like our approach, their method is based on complex roots of unity and achieves a reduction in multiplicative depth compared to the original CKKS bootstrapping.

The main difference is that they focus on a full $N/2$ -slots bootstrapping using a large number of processors, while we focus on efficient bootstrapping for a small number of slots, with a

single or few processors. Additionally, the underlying techniques are fundamentally different. Their method homomorphically evaluates the polynomial shift arising in the decryption process through a blind rotation procedure — inspired by the DM/CGGI bootstrapping technique. This involves encrypting the index j of each of the h non-zero secret-key elements (where h is the Hamming weight), and then performing a blind rotation by j positions on a vector of complex roots of unity. In contrast, our method does not encode the secret-key coefficients by their indices. Instead, the secret-key coefficients are packed directly into the slots of a CKKS ciphertext, and bootstrapping is performed by homomorphically evaluating the decryption equation for each slot over the complex roots of unity.

2 Preliminaries

Basic notations. For a real number r , we denote by $\lceil r \rceil$ the nearest integer to r , rounding upwards. We denote vectors in bold, and $\langle \mathbf{a}, \mathbf{b} \rangle$ denotes the scalar product of the vectors $\mathbf{a}$ and $\mathbf{b}$. For an integer q , we use $\mathbb{Z} \cap (-q/2, q/2]$ as a representative of $\mathbb{Z}_q$, and use $[z]_q$ or $z \bmod q$ to denote the central reduction of the integer z modulo q into that interval. For a finite set S , we denote by $v \leftarrow S$ the sampling of v uniformly at random from S . By $v \leftarrow \mathcal{D}$ we also denote sampling v from a distribution $\mathcal{D}$. We utilize the same notations for sampling coefficients of vectors/polynomials independently and identically from a set/distribution. For an integer $\kappa > 0$, we let $v \leftarrow \chi_\kappa$ be sampled from the *centered binomial distribution* χ_κ . It is defined as outputting $v := \sum_{j=1}^\kappa (a_j - b_j)$, where $\mathbf{a}, \mathbf{b} \leftarrow \{0, 1\}^\kappa$. In this case, it holds that $|v| \leq \kappa$. We denote by λ the security parameter; all known attacks should take $\Omega(2^\lambda)$ bit operations.

Cyclotomic rings. For a power-of-two N , we set $\mathcal{R} := \mathbb{Z}[X]/(X^N + 1)$ as the $2N$ -th cyclotomic ring. We also write $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^N + 1)$ for the residue ring of $\mathcal{R}$ modulo q . As previously, coefficients of elements of $\mathcal{R}_q$ are represented as integers in $(-q/2, q/2]$. We represent an arbitrary element of the cyclotomic ring $\mathcal{S} = \mathbb{R}[X]/(X^N + 1)$ as a polynomial $a(X) = \sum_{j=0}^{N-1} a_j X^j$ of degree strictly less than N , and identify it with its coefficient vector $(a_0, \dots, a_{N-1}) \in \mathbb{R}^N$. We consider the norms $\|a\|_\infty = \max_{i \in \{0, \dots, N-1\}} |a_i|$ and $\|a\|_1 = \sum_{i=0}^{N-1} |a_i|$ on the coefficient vector of a .

3 The CKKS scheme

3.1 Basic RLWE-based encryption

In [CKKS17], the authors described a fully homomorphic encryption scheme designed for approximate arithmetic. Given ciphertexts encrypting m_1 and m_2 , the scheme enables secure computations of encrypted approximate values of $m_1 + m_2$ and $m_1 \cdot m_2$, with a predefined level of precision. We recall the concrete instantiation based on the BGV scheme [BGV11], using the multiplication from [GHS12b] based on raising the ciphertext modulus. Note that the CKKS scheme differs from BGV in the sense that the plaintext space is the polynomial ring $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, instead of $\mathbb{Z}_t[X]/(X^N + 1)$ for a small t .

Let q be an integer and let s be a secret key with small components in $\mathcal{R}$. A CKKS ciphertext $\text{ct} = (c_0, c_1)$ for a plaintext $m \in \mathcal{R}$ satisfies:

$$\langle \text{ct}, \text{sk} \rangle = c_0 + c_1 \cdot s = m + e \pmod{(q, X^N + 1)}$$

for some small error $e \in \mathcal{R}$. This implies that during decryption, the message m is not recovered exactly, as the low-order bits of the coefficients of $m(X)$ are masked by the error $e(X)$. Such an error e is inserted to guarantee the security of the hardness assumption of the ring learning with error (RLWE) problem. During homomorphic operations, the coefficients of the plaintext polynomial $m(X)$ should remain sufficiently small compared to the ciphertext modulus q .

Definition 1 (RLWE). For an integer q , a ring dimension N and distributions $\mathcal{D}_s$ and $\mathcal{D}_e$, the decisional RLWE problem consists, for $s \leftarrow \mathcal{D}_s$, in distinguishing polynomially many samples $(a, a \cdot s + e) \in \mathcal{R}_q^2$ from uniform in $\mathcal{R}_q^2$, where $a \leftarrow \mathcal{R}_q$ and $e \leftarrow \mathcal{D}_e$.

We now formally describe the scheme, following [CHK⁺18a], beginning with the construction of a *leveled* homomorphic encryption scheme for approximate arithmetic. The plaintext space is $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, while the ciphertext space is $\mathcal{R}_q^2$. The scheme defines various layers of moduli, i.e. $q_\ell = q_0 \cdot \Delta^\ell$ for $1 \leq \ell \leq L$ and a base Δ . For the error sampling, for simplicity we use the centered binomial distribution χ_κ with parameter κ , rather than a discrete Gaussian distribution. For $h \in \mathbb{N}$, we let $\mathcal{HW}(h)$ be the set of ternary vectors in $\{0, \pm 1\}^N$ whose Hamming weight is exactly h . For a real $0 \leq \rho \leq 1$, let $\mathcal{ZO}(\rho)$ denote the distribution that outputs a vector in $\{0, \pm 1\}^N$, where each entry is sampled independently, with probability $\rho/2$ for both -1 and 1 , and probability $1 - \rho$ for 0 .

- **KeyGen(1^λ):**
 - For a base Δ , a base modulus q_0 and an integer L , let $q_\ell = q_0 \cdot \Delta^\ell$ for $\ell = 1, \dots, L$. Given the security parameter λ , choose a power-of-two N , an integer P , and an integer $\kappa > 0$ for an RLWE problem that achieves a security level of λ -bits, for a modulus $P \cdot q_L$ and a ring degree N .
 - Sample $s \leftarrow \mathcal{HW}(h)$, $a \leftarrow \mathcal{R}_{q_L}$ and $e \leftarrow \chi_\kappa$. Set the secret key as $\text{sk} \leftarrow (1, s)$ and the public key as $\text{pk} \leftarrow (b, a) \in \mathcal{R}_{q_L}^2$ where $b = -as + e \pmod{q_L}$.
- **KSGen_{sk}(s'):** for $s' \in \mathcal{R}$, sample $a \leftarrow \mathcal{R}_{P \cdot q_L}$ and $e' \leftarrow \chi_\kappa$. Return the switching key as $\text{swk} \leftarrow (b', a') \in \mathcal{R}_{P \cdot q_L}$ where $b' = -a's + e' + Ps' \pmod{P \cdot q_L}$.
 - Set the evaluation key as $\text{evk} \leftarrow \text{KSGen}_{\text{sk}}(s^2)$.
- **Enc_{pk}(m):** for $m \in \mathcal{R}$, sample $v \leftarrow \mathcal{ZO}(1/2)$ and $e_0, e_1 \leftarrow \chi_\kappa$. Output $\text{ct} = v \cdot \text{pk} + (m + e_0, e_1) \pmod{q_L}$.
- **Dec_{sk}(ct):** for $\text{ct} = (c_0, c_1) \in \mathcal{R}_{q_\ell}^2$, output $m = c_0 + c_1 \cdot s \pmod{q_\ell}$.

We now define the homomorphic operations $\text{Add}(\text{ct}_1, \text{ct}_2)$ and $\text{Mult}_{\text{evk}}(\text{ct}_1, \text{ct}_2)$. We also define the more lightweight external multiplication ExtMult , which multiplies a ciphertext by a plaintext $v \in \mathcal{R}$.

- **Add(ct_1, ct_2):** for $\text{ct}_1, \text{ct}_2 \in \mathcal{R}_{q_\ell}^2$, output $\text{ct}_{\text{add}} = \text{ct}_1 + \text{ct}_2 \pmod{q_\ell}$.
- **Mult_{evk}(ct_1, ct_2):** for $\text{ct}_1 = (b_1, a_1)$, $\text{ct}_2 = (b_2, a_2) \in \mathcal{R}_{q_\ell}^2$, let $(d_0, d_1, d_2) = (b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2) \pmod{q_\ell}$. Output $\text{ct}_{\text{mult}} \leftarrow (d_0, d_1) + \lfloor P^{-1} \cdot d_2 \cdot \text{evk} \rfloor \pmod{q_\ell}$.
- **ExtMult(ct, v):** for $\text{ct} = (b, a) \in \mathcal{R}_{q_\ell}^2$ and $v \in \mathcal{R}$, output $\text{ct}_{\text{extmult}} = (b \cdot v, a \cdot v) \pmod{q_\ell}$.

The rescaling procedure below transforms an encryption of m modulo q into an encryption of m/Δ , while also scaling its inherent noise e to around e/Δ . As will be seen in Section 3.2, the composition of homomorphic multiplication and rescaling enables to mimic fixed-point arithmetic, while managing the error growth.

- **RS $_{\ell \rightarrow \ell'}$ (ct):** for a ciphertext $\text{ct} \in \mathcal{R}_{q_\ell}^2$ at level ℓ , output $\text{ct}' = \lfloor \Delta^{\ell' - \ell} \cdot \text{ct} \rfloor \pmod{q_{\ell'}}$. We omit the subscript $\ell \rightarrow \ell'$ when $\ell' = \ell - 1$.

RNS arithmetic. In practice, efficient implementations of CKKS use the so-called RNS-CKKS variant [CHK⁺18b], which relies on the Residue Number System (RNS) representation. In this setting, the ciphertext moduli have the form $Q_r = \prod_{0 \leq i \leq r} q_i$, where each q_i is a small (word-size) prime and $0 \leq r \leq \ell_m$.

Noise growth. In [CKKS17], the authors state some lemmata for estimating the noise, although their noise estimation is only heuristic. Our lemmata below are with rigorous proofs, though with larger upper bounds; we refer to Appendix A for the proofs.

Lemma 1. *Let $\text{ct} \leftarrow \text{Enc}_{\text{pk}}(m)$ be an encryption of $m \in \mathcal{R}$. Then $\langle \text{ct}, \text{sk} \rangle = m + e_{\text{clean}} \pmod{q_L}$ for some clean encryption error $e_{\text{clean}} \in \mathcal{R}$ satisfying $\|e_{\text{clean}}\|_{\infty} \leq B_{\text{clean}} := 3N\kappa$.*

Lemma 2. *Let $\text{ct}' \leftarrow \text{RS}_{\ell \rightarrow \ell'}(\text{ct})$ for a ciphertext $\text{ct} \in \mathcal{R}_{q_\ell}^2$. Then $\langle \text{ct}', \text{sk} \rangle = \lfloor \langle \text{ct}, \text{sk} \rangle \cdot q_{\ell'} / q_\ell \rfloor + e_{\text{rs}} \pmod{q_{\ell'}}$ for some rescaling error $e_{\text{rs}} \in \mathcal{R}$ satisfying $\|e_{\text{rs}}\|_{\infty} \leq B_{\text{rs}} := 2N$.*

Lemma 3. *Let $\text{ct}_{\text{mult}} \leftarrow \text{Mult}_{\text{evk}}(\text{ct}_1, \text{ct}_2)$ for two ciphertexts $\text{ct}_1, \text{ct}_2 \in \mathcal{R}_{q_\ell}^2$. If $P \geq Nq_\ell\kappa$, then $\langle \text{ct}_{\text{mult}}, \text{sk} \rangle = \langle \text{ct}_1, \text{sk} \rangle \cdot \langle \text{ct}_2, \text{sk} \rangle + e_{\text{mult}} \pmod{q_\ell}$ for some e_{mult} satisfying $\|e_{\text{mult}}\|_{\infty} \leq B_{\text{mult}} := 2N$.*

Writing $\langle \text{ct}_1, \text{sk} \rangle = m_1 + e_1 \pmod{q_\ell}$ and $\langle \text{ct}_2, \text{sk} \rangle = m_2 + e_2 \pmod{q_\ell}$, the previous lemma shows that only the high-order bits of $m_1 m_2$ can be recovered, as we get $\langle \text{ct}_{\text{mult}}, \text{sk} \rangle = (m_1 + e_1) \cdot (m_2 + e_2) + e_{\text{mult}} = m_1 m_2 + e^* \pmod{q_\ell}$, and the low-order bits of $m_1 m_2$ are masked by the quite large error $e^* = e_1 m_2 + e_2 m_1 + e_1 e_2 + e_{\text{mult}}$. Applying Lemma 2, one can then perform a modulus switching to rescale the product, which then gives:

$$\langle \text{ct}'_{\text{mult}}, \text{sk} \rangle = \lfloor m_1 m_2 / \Delta \rfloor + e'_{\text{mult}} \pmod{q_{\ell-1}}.$$

Therefore, one obtains an encryption of the scaled product $\lfloor m_1 m_2 / \Delta \rfloor$.

3.2 Encrypting single real numbers

For simplicity, we first recall the CKKS encryption of single real numbers. We then recall the encryption of $N/2$ complex numbers in parallel in Section 3.3.

Specifically, the CKKS scheme can encrypt a real value $x \in \mathbb{R}$ with a specified precision, by rounding to an integer after scaling. This enables to define (approximate) homomorphic operations directly over $\mathbb{R}$. More precisely, to encode a real value $x \in \mathbb{R}$ with precision Δ , we use

$$\text{Ecd}(x) = \lfloor x \cdot \Delta \rfloor. \tag{2}$$

Therefore, to encrypt $x \in \mathbb{R}$, we first encode x into $m = \text{Ecd}(x)$, and then encrypt m into $\text{ct} = \text{Enc}_{\text{pk}}(m)$, which gives $\langle \text{ct}, \text{sk} \rangle = \text{Ecd}(x) + e \pmod{q_L}$. Since $\text{Ecd}(x) \in \mathbb{Z}$, we are using only a single coefficient of the plaintext space $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$. Formally, we define the following encryption procedure for real numbers, and the homomorphic multiplication of two ciphertexts of real numbers.

- $\text{EncR}_{\text{pk}}(x)$: for $x \in \mathbb{R}$, return $\text{ct} \leftarrow \text{Enc}_{\text{pk}}(\text{Ecd}(x))$.
- $\text{MultR}_{\text{evk}}(\text{ct}_1, \text{ct}_2)$: return $\text{RS}(\text{Mult}_{\text{evk}}(\text{ct}_1, \text{ct}_2))$
- $\text{ExtMultR}(\text{ct}, v)$: return $\text{RS}(\text{ExtMult}(\text{ct}, v))$

Thanks to the rescaling by a factor Δ , given as input two ciphertexts ct_1, ct_2 encrypting $x_1, x_2 \in \mathbb{R}$ under a modulus q_ℓ , we can obtain a new ciphertext $\text{ct} = \text{MultR}_{\text{evk}}(\text{ct}_1, \text{ct}_2)$ encrypting $\text{Ecd}(x_1) \cdot \text{Ecd}(x_2)/\Delta \simeq \text{Ecd}(x_1 \cdot x_2)$, therefore an encryption of the product $x_1 \cdot x_2 \in \mathbb{R}$. However, the new encryption is under a smaller modulus $q_{\ell-1} = q_\ell/\Delta$. This is captured in the following lemma; see Fig. 2 for an illustration.

Lemma 4. *Let $\text{ct}_{\text{multR}} \leftarrow \text{MultR}_{\text{evk}}(\text{ct}_1, \text{ct}_2)$ for two ciphertexts $\text{ct}_1, \text{ct}_2 \in \mathcal{R}_{q_\ell}^2$ such that $\langle \text{ct}_i, \text{sk} \rangle = \text{Ecd}(x_i) + e_i \pmod{q_\ell}$ for $i = 1, 2$, for $x_i \in \mathbb{R}$ with $|x_i| \leq \nu < N$, and $\|e_i\|_\infty \leq E$, where $E^2 \leq \Delta$. Then $\langle \text{ct}_{\text{multR}}, \text{sk} \rangle = \text{Ecd}(x_1 \cdot x_2) + e_{\text{multR}} \pmod{q_{\ell-1}}$ for some e_{multR} satisfying $\|e_{\text{multR}}\|_\infty \leq 2\nu E + 8N$.*

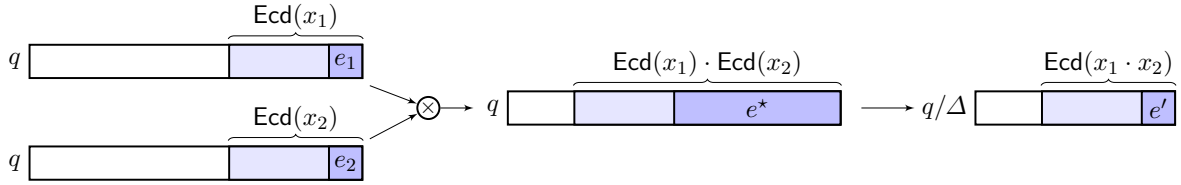


Fig. 2. Homomorphic multiplication of real numbers in CKKS: given two encryptions of $x_1, x_2 \in \mathbb{R}$, we obtain an encryption of $x_1 \cdot x_2 \in \mathbb{R}$, albeit for a smaller modulus q/Δ .

3.3 Packing method for parallel encryption of complex numbers

In [CKKS17,CHK⁺18a], the authors described a packing method to encrypt multiple messages in a single ciphertext. Through a ring isomorphism, the set $\mathcal{S} = \mathbb{R}[X]/(X^N + 1)$ can be identified with the complex coordinate space $\mathbb{C}^{N/2}$. This enables to encrypt $N/2$ complex numbers in a single ciphertext, and perform parallel operations in a SIMD manner.

More precisely, the authors considered a variant of the Fourier transform

$$\text{DFT} : \mathbb{R}[X]/(X^N + 1) \rightarrow \mathbb{C}^{N/2}, \quad p \mapsto \left(p(\zeta^{5^0}), p(\zeta^{5^1}), \dots, p(\zeta^{5^{N/2-1}}) \right)$$

where ζ is a primitive $(2N)$ -th root of unity. We denote by $\text{iDFT} : \mathbb{C}^{N/2} \rightarrow \mathcal{S}$ the inverse transform. The DFT function is a ring isomorphism from the ring $\mathcal{S}$ (the coefficients space) to the ring of complex vectors in $\mathbb{C}^{N/2}$ (the slots space). Its purpose is therefore to map polynomial addition/multiplication in the ring $\mathcal{S}$ to component-wise addition/multiplication in $\mathbb{C}^{N/2}$.

Encoding. For RLWE-type schemes, we must encode our $N/2$ complex values into polynomials with integer coefficients in the plaintext ring $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, instead of $\mathbb{R}[X]/(X^N + 1)$. Therefore, the encoding map $\text{Ecd} : \mathbb{C}^{N/2} \rightarrow \mathcal{R}$ uses an integer approximation of iDFT , with a scaling factor Δ :

$$\forall \mathbf{z} \in \mathbb{C}^{N/2} : \text{Ecd}(\mathbf{z}) = \lfloor \Delta \cdot \text{iDFT}(\mathbf{z}) \rfloor$$

and the corresponding decoding map $\text{Dcd} : \mathcal{R} \rightarrow \mathbb{C}^{N/2}$ is defined as $\forall p \in \mathcal{R} : \text{Dcd}(p) = \frac{1}{\Delta} \cdot \text{DFT}(p)$. We denote by $\times$ the scaled multiplication over $\mathcal{R}$, i.e. $p_1 \times p_2 = \lfloor p_1 \cdot p_2 / \Delta \rfloor$ for any $p_1, p_2 \in \mathcal{R}$. We then have the approximate homomorphic properties:

$$\begin{aligned} \text{Ecd}(\mathbf{z}_1 + \mathbf{z}_2) &\simeq \text{Ecd}(\mathbf{z}_1) + \text{Ecd}(\mathbf{z}_2) \\ \text{Ecd}(\mathbf{z}_1 \odot \mathbf{z}_2) &\simeq \text{Ecd}(\mathbf{z}_1) \times \text{Ecd}(\mathbf{z}_2) \end{aligned}$$

where the vectors $\mathbf{z}_1$ and $\mathbf{z}_2$ are added and multiplied component-wise. Therefore, to perform parallel addition/multiplication on $N/2$ complex values, we can first encode them as a polynomial $p(X)$ using Ecd , and then perform ring addition / multiplication in $\mathcal{R}$. Eventually, we can decode the polynomial into $N/2$ complex values using Dcd ; see Fig. 3 for an illustration.

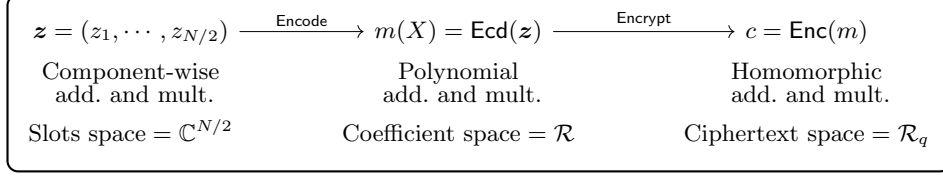


Fig. 3. Packing method for CKKS.

Sparse packing. For efficiency reasons, it can be advantageous to encode fewer than $N/2$ components. We consider a power-of-two n dividing $N/2$. The sub-ring $\mathbb{R}[X^{N/(2n)}]/(X^N + 1) \simeq \mathbb{R}[Y]/(Y^{2n} + 1)$ is isomorphic to $\mathbb{C}^n$. In particular, if $p \in \mathbb{R}[X^{N/(2n)}]/(X^N + 1)$, then $\text{DFT}(p)$ contains $N/(2n)$ repetitions of the same n components. We can therefore extend the encoding map Ecd to encode vectors of n complex values instead of $N/2$, as follows:

$$\forall \mathbf{z} \in \mathbb{C}^n : \text{Ecd}(\mathbf{z}) = \lfloor \Delta \cdot \text{iDFT}(\underbrace{\mathbf{z}, \dots, \mathbf{z}}_{N/(2n) \text{ rep.}}) \rfloor \in \mathbb{Z}[X^{N/(2n)}]/(X^N + 1). \quad (3)$$

In particular, for $n = 1$, we have $\text{Ecd}(z) = \lfloor \Delta \cdot \text{Re}(z) \rfloor + X^{N/2} \cdot \lfloor \Delta \cdot \text{Im}(z) \rfloor$ for any $z \in \mathbb{C}$. Therefore, the encoding map Ecd defined in (3) is a generalization of the encoding function defined in (2) for encoding $x \in \mathbb{R}$.

Message packing into a ciphertext. Thanks to this generalized encoding function for any $\mathbf{z} \in \mathbb{C}^n$, we can extend the encryption procedure $\text{EncR}_{\text{pk}}(x)$ from Section 3.2 to any $\mathbf{z} \in \mathbb{C}^n$, for any power-of-two $1 \leq n \leq N/2$:

- $\text{EncR}_{\text{pk}}(\mathbf{z})$: for $\mathbf{z} \in \mathbb{C}^n$, return $\text{ct} \leftarrow \text{Enc}_{\text{pk}}(\text{Ecd}(\mathbf{z}))$.

To perform a homomorphic multiplication, we employ the same rescaled multiplication procedure $\text{MultR}_{\text{evk}}(\text{ct}_1, \text{ct}_2)$ as in the previous section. Given ciphertexts $\text{ct}_1 \leftarrow \text{EncR}_{\text{pk}}(\mathbf{z}_1)$ and $\text{ct}_2 \leftarrow \text{EncR}_{\text{pk}}(\mathbf{z}_2)$, when applying $\text{Add}(\text{ct}_1, \text{ct}_2)$ and $\text{MultR}_{\text{evk}}(\text{ct}_1, \text{ct}_2)$, the vectors $\mathbf{z}_1, \mathbf{z}_2 \in \mathbb{C}^n$ are homomorphically added and multiplied component-wise; see Figure 3 for an illustration.

3.4 Rotation and conjugation of the slots

As demonstrated in [CKKS17], rotation and conjugation of slots can be performed homomorphically by applying an automorphism Ψ to both components of a ciphertext. However, Ψ is also applied to the secret key, meaning the new ciphertext only decrypts correctly with the modified key $s' = \Psi(s)$. Therefore, a key-switching procedure is required to revert to the original key s .

Key switching. The goal of key switching is to convert a ciphertext under a secret key $\text{sk}' = (1, s')$ into a ciphertext for the same message with respect to another secret key $\text{sk} = (1, s)$. The switching key is generated by the procedure $\text{swk} \leftarrow \text{KSGen}_{\text{sk}}(s')$. Formally, the key switching proceeds as follows:

- $\text{KS}_{\text{swk}}(\text{ct})$: for $\text{ct} = (c_0, c_1)$, return $\text{ct}' \leftarrow (c_0, 0) + \lfloor P^{-1} \cdot c_1 \cdot \text{swk} \rfloor \pmod{q_\ell}$.

The noise growth from key switching is captured in the following lemma.

Lemma 5 (Key switching). *Let $\text{ct} = (c_0, c_1) \in \mathcal{R}_{q_\ell}^2$ be a ciphertext with respect to a secret key $\text{sk}' = (1, s')$ and let $\text{swk} \leftarrow \text{KSGen}_{\text{sk}}(s')$. If $P \geq Nq_\ell\kappa$, then $\text{ct}' \leftarrow \text{KS}_{\text{swk}}(\text{ct})$ satisfies $\langle \text{ct}', \text{sk} \rangle = \langle \text{ct}, \text{sk}' \rangle + e_{\text{ks}} \pmod{q_\ell}$ for some $e_{\text{ks}} \in \mathcal{R}$ with $\|e_{\text{ks}}\|_\infty \leq 2N$.*

Rotation. For $\mathcal{S} = \mathbb{R}[X]/(X^N + 1)$, the automorphism $\Psi_r : \mathcal{S} \rightarrow \mathcal{S}$, given $m(X) \in \mathcal{S}$, returns $m(X^{5^r}) \pmod{X^N + 1}$. It induces a rotation by r positions to the left on the vector of encoded values. Namely for any $m(X) \in \mathcal{S}$, letting $\text{DFT}(m) = (z_j)_{0 \leq j < N/2}$, we get:

$$\begin{aligned} \text{DFT}(\Psi_r(m)(X)) &= (\Psi_r(m)(\zeta^{5^j}))_{0 \leq j < N/2} = (m((\zeta^{5^j})^{5^r}))_{0 \leq j < N/2} \\ &= (m(\zeta^{5^{r+j}}))_{0 \leq j < N/2} = (z_{r+j \bmod N/2})_{0 \leq j < N/2}. \end{aligned}$$

Conjugation. We also consider the automorphism $\text{conj} : \mathcal{S} \rightarrow \mathcal{S}$ defined as $\text{conj}(m)(X) = m(X^{-1}) \pmod{X^N + 1}$. It enables to compute the conjugate of the values encoded into a polynomial, namely, we have that $\text{DFT}(\text{conj}(m)) = \overline{\text{DFT}(m)}$. We also denote by $\text{Im2} : \mathcal{S} \rightarrow \mathcal{S}$ the operator

$$\text{Im2}(m) = -X^{N/2} \cdot (m - \text{conj}(m)).$$

Since $-\Delta \cdot X^{N/2} = \text{Ecd}(1/i)$, given any $z \in \mathbb{C}^{N/2}$, we have $\text{DFT}(\text{Im2}(\text{iDFT}(z))) = 2 \cdot \text{Im}(z)$. Therefore, the Im2 operator enables to extract the imaginary part of the slots, up to a factor 2. Similarly, we define the Re2 operator with $\text{Re2}(m) = \text{conj}(m) + m$, so that $\text{DFT}(\text{Re2}(\text{iDFT}(z))) = 2 \cdot \text{Re}(z)$.

Homomorphic rotation/conjugation of the slots. We summarize below the procedures for homomorphic rotation/conjugation of the slots for a ciphertext $\text{ct} = (c_0, c_1)$:

- $\text{GenRot}_{\text{sk}}(r)$: generate the rotation key $\text{rk}_r \leftarrow \text{KSGen}_{\text{sk}}(\Psi_r(s))$.
- $\text{Rot}(\text{ct}, r)$: return $\text{KS}_{\text{rk}_r}((\Psi_r(c_0), \Psi_r(c_1)))$.
- $\text{GenConj}_{\text{sk}}$: generate the conjugation key $\text{ck} \leftarrow \text{KSGen}_{\text{sk}}(\text{conj}(s))$.
- $\text{Conj}(\text{ct})$: return $\text{KS}_{\text{ck}}((\text{conj}(c_0), \text{conj}(c_1)))$.
- $\text{Im2}(\text{ct})$: return the ciphertext $\text{ExtMult}(\text{Conj}(\text{ct}) - \text{ct}, X^{N/2})$.

3.5 The trace and product operators

Summing the slots. We recall how to homomorphically compute partial or full summation of the slots. The sum of all slots could be obtained by repeatedly rotating the slots by one position and then accumulating the result by summing. However, this would require $N/2 - 1$ rotations for $N/2$ slots, which would be impractical. In the following, we recall the technique from [CHK⁺18a, Alg. 2], which uses only $\mathcal{O}(\log N)$ homomorphic operations. Summing all slots is algebraically known as the field trace operator [CDKS20].

Formally, for a power-of-two integer a dividing $N/2$, using the Ψ_r automorphism from the previous section, we define the operator $\text{Tr}_{N/2 \rightarrow a}$ as

$$\text{Tr}_{N/2 \rightarrow a} = (\text{id} + \Psi_a) \circ (\text{id} + \Psi_{2a}) \circ (\text{id} + \Psi_{4a}) \circ \dots \circ (\text{id} + \Psi_{N/4}).$$

We show in Appendix B.1 that the $\text{Tr}_{N/2 \rightarrow a}$ operator computes partial sums of the slots sharing the same index modulo a :

$$\text{Tr}_{N/2 \rightarrow a}(\text{iDFT}(z_0, \dots, z_{N/2-1})) = \text{iDFT}(\underbrace{t, \dots, t}_{N/(2a) \text{ rep.}})$$

for $\mathbf{t} = (t_k)_{0 \leq k < a}$, with $t_k = z_k + z_{k+a} + \dots + z_{k+(N/(2a)-1) \cdot a}$. Therefore, the $\text{Tr}_{N/2 \rightarrow a}$ operator outputs an encoding of the a slots $t_0, \dots, t_{a-1}$, which are repeated $N/(2a)$ times to get $N/2$ slots. The $\text{Tr}_{N/2 \rightarrow a}$ operator can be homomorphically evaluated, requiring $\log_2(N/(2a))$ automorphisms and key switchings, while consuming no multiplicative levels; see Appendix B.1.

Multiplying the slots. Similarly to the trace operator, we can define the product operator to compute the partial or full product of all slots, by replacing sums by scaled products in the definition. For powers-of-two $b < a \leq N/2$, we define the $\text{Pr}_{a \rightarrow b}$ operator as

$$\text{Pr}_{a \rightarrow b} := (\text{id} \times \Psi_b) \circ (\text{id} \times \Psi_{2b}) \circ (\text{id} \times \Psi_{4b}) \circ \dots \circ (\text{id} \times \Psi_{a/2})$$

where as in Section 3.3 we denote by $\times$ the scaled multiplication over $\mathcal{R}$, with $p_1 \times p_2 = \lfloor p_1 \cdot p_2 / \Delta \rfloor$ for any $p_1, p_2 \in \mathcal{R}$. Letting $\mathbf{z} \in \mathbb{C}^a$, the $\text{Pr}_{a \rightarrow b}$ operator computes partial products of the slots sharing the same index modulo b :

$$\text{Pr}_{a \rightarrow b}(\text{Ecd}(\mathbf{z})) \simeq \text{Ecd}(\mathbf{t})$$

where $\mathbf{t} = (t_k)_{0 \leq k < b}$, with $t_k = z_k \cdot z_{k+b} \cdots z_{k+(a/b-1) \cdot b}$ for $0 \leq k < b$. The $\text{Pr}_{a \rightarrow b}$ operator can be homomorphically evaluated using $\log_2(a/b)$ automorphisms and multiplications; thus, it consumes $\log_2(a/b)$ levels; see Appendix B.2. Note that the product operator $\text{Pr}_{a \rightarrow b}$ corresponds to the standard field norm of $\mathbb{Q}(\zeta_a)$ over $\mathbb{Q}(\zeta_b)$, where ζ_k denotes a primitive k -th root of unity.

3.6 From slots back to polynomial coefficients

Our bootstrapping algorithm for multiple slots requires putting the slots back to the coefficients of a polynomial; this is the `SlotToCoeff` operation. The original CKKS bootstrapping requires both `CoeffToSlot` and `SlotToCoeff` operations [CHK⁺18a], while our technique only requires `SlotToCoeff`, so we only describe the latter; the two operations are the inverse of each other.

Formally, `SlotToCoeff` _{n} takes as input a polynomial $v(X)$ encoding n slots $\mathbf{t} = (t_0, \dots, t_{n-1}) \in \mathbb{R}^n$:

$$v(X) = \text{Ecd}(t_0, \dots, t_{n-1}) \in \mathbb{Z}[X^{N/(2n)}]/(X^N + 1)$$

and outputs a polynomial $m(X) \in \mathbb{Z}[X^{N/n}]/(X^N + 1)$ whose n coefficients are the t_j 's, scaled by a factor $\Delta \in \mathbb{Z}$:

$$m(X) = \text{SlotToCoeff}_n(v) = \sum_{j=0}^{n-1} \lfloor \Delta \cdot t_j \rfloor \cdot X^{j \cdot N/n} \quad (4)$$

Since `SlotToCoeff` is a linear map, it can be homomorphically evaluated as a matrix-vector multiplication, requiring $\mathcal{O}(n)$ homomorphic operations [CHK⁺18a]. However, the homomorphic Discrete Fourier Transform (DFT), which is the primary computation within `SlotToCoeff`, can be performed more efficiently with only $\mathcal{O}(\log n)$ homomorphic operations [CHH18, CCS19]. In this case, the polynomial coefficients in (4) are recovered in a bit-reversed order. We refer to Appendix D for a detailed description.

4 New bootstrapping equation for CKKS: single slot

We now introduce our new bootstrapping algorithm, which is quite different from the original CKKS bootstrapping recalled in the previous section. For simplicity, we first consider a CKKS ciphertext ct encrypting a single slot, that is $\langle \text{ct}, \text{sk} \rangle = m + e \pmod{q}$ for $m \in \mathbb{Z}$ and $e \in \mathcal{R}$. We will describe the bootstrapping of $n \geq 2$ slots in parallel in Section 5.

An RLWE-format ciphertext can be viewed as (multiple) LWE-format ciphertexts. Therefore, we first convert $\text{ct} = (b, a)$ into an LWE ciphertext $\mathbf{c} \in \mathbb{Z}_q^N$ for the same $m \in \mathbb{Z}$, i.e. we set $\mathbf{c} = (a_0 + b_0, -a_{N-1}, -a_{N-2}, \dots, -a_1)$ and $\mathbf{s} = (s_0, \dots, s_{N-1})$, assuming that $s_0 = 1$. LWE decryption now comprises computing the dot product $\langle \mathbf{c}, \mathbf{s} \rangle = m + e_0 \pmod{q}$ for $e_0 \in \mathbb{Z}$. Unlike the original CKKS bootstrapping, which uses sparse ternary secrets, we first assume that $\mathbf{s}$ is a random binary vector, not necessarily constrained to a low Hamming weight.

4.1 Our new bootstrapping equation

Our new bootstrapping equation consists in embedding the additive group $\mathbb{Z}_q$ into the multiplicative circle group of complex numbers, using the homomorphism:

$$f : \mathbb{Z}_q \rightarrow \mathbb{C}, \quad x \mapsto \exp(2i\pi \cdot x/q).$$

We can therefore compute the decryption equation $m = \langle \mathbf{s}, \mathbf{c} \rangle = \sum_{k=0}^{N-1} s_k c_k \pmod{q}$ over the multiplicative circle group in $\mathbb{C}$:

$$\exp(2i\pi \cdot m/q) = \prod_{k=0}^{N-1} \exp(2i\pi \cdot c_k s_k / q)$$

Moreover, since $s_k \in \{0, 1\}$ works as a selector, we can write:

$$\exp(2i\pi \cdot m/q) = \prod_{k=0}^{N-1} (1 + (\exp(2i\pi \cdot c_k/q) - 1) \cdot s_k). \quad (5)$$

By isolating the imaginary part and assuming $m \ll q$, we obtain:

$$\frac{2\pi \cdot m}{q} \simeq \sin\left(\frac{2\pi \cdot m}{q}\right) = \text{Im} \left(\prod_{k=0}^{N-1} (1 + (\exp(2i\pi \cdot c_k/q) - 1) \cdot s_k) \right). \quad (6)$$

We want to obtain an encryption of $m \in \mathbb{Z}$, which corresponds to an encoding of $m/\Delta \in \mathbb{R}$. Therefore, we scale the above equation by a factor $q/(2\pi\Delta)$, and we use the approximation:

$$\frac{q}{2\pi\Delta} \cdot \sin\left(\frac{2\pi m}{q}\right) = \frac{q}{2\pi\Delta} \cdot \left(\frac{2\pi m}{q} + \mathcal{O}\left(\frac{m^3}{q^3}\right) \right) = \frac{m}{\Delta} + \mathcal{O}\left(\frac{m^3}{q^2\Delta}\right). \quad (7)$$

Eventually, we obtain from (6):

$$\frac{m}{\Delta} \simeq \frac{q}{2\pi\Delta} \cdot \text{Im} \left(\prod_{k=0}^{N-1} (1 + (\exp(2i\pi \cdot c_k/q) - 1) \cdot s_k) \right). \quad (8)$$

From (7), the above approximation requires $m = \mathcal{O}(q^{2/3})$, the same condition as in the original CKKS bootstrapping.

Bootstrapping. We claim that (8) provides a new bootstrapping equation for CKKS, as it can be homomorphically evaluated to provide a new ciphertext for the same $m \in \mathbb{Z}$, but under a larger modulus $q \cdot p$ than the original modulus q . For this, we stress that in the above equation, we consider all variables over $\mathbb{R}$, and since $\exp(2i\pi \cdot c_k/q) \in \mathbb{C}$, we treat its real and imaginary part separately. Each complex multiplication decomposes into 4 multiplications and 2 additions over $\mathbb{R}$. Since a complex addition also decomposes into two real additions, we can work entirely over $\mathbb{R}$. Similarly, we view the final scaling as a multiplication by $q/(2\pi\Delta) \in \mathbb{R}$, and the final output $m/\Delta \in \mathbb{R}$.

Correspondingly, for the homomorphic evaluation of (8), we consider all ciphertexts as encryptions of real numbers, using the encryption function $\text{EncR}(x)$ for $x \in \mathbb{R}$ introduced in Section 3.2. Consequently, all homomorphic operations are performed on encryptions of reals. Eventually, since $m/\Delta \in \mathbb{R}$ is encoded as $m = \text{Ecd}(m/\Delta) \in \mathbb{Z}$, the encryption of m/Δ will correspond to an encryption of the original plaintext $m \in \mathbb{Z}$, but under a larger modulus $q \cdot p$.

Therefore, for bootstrapping, we assume that we are given encryptions $S_k \leftarrow \text{EncR}_{\text{pk}}(s_k)$ of each bit $s_k \in \{0, 1\}$ of the secret key $\mathbf{s}$. Here, although $s_k \in \{0, 1\}$, we again view $s_k \in \mathbb{R}$. Then, by homomorphic multiplications (using ExtMultR) and homomorphic additions, we obtain encryptions of $1 + (\exp(2i\pi \cdot c_k/q) - 1) \cdot s_k$ for each $0 \leq k < N$, while still encrypting the real/imaginary parts separately. The product of the N terms can then be homomorphically computed with a tree of multiplicative depth $\log_2 N$, such that, according to (5), we obtain an encryption of $\exp(2i\pi \cdot m/q)$. Eventually, we only keep the encryption of its imaginary part and scale by $q/(2\pi\Delta)$. According to (8), this provides an encryption of the real m/Δ , or equivalently, an encryption of the integer $m = \text{Ecd}(m/\Delta)$.

The circuit corresponding to (8) has a total multiplicative depth of $\ell = \log_2(N) + 2$. Thus, starting with a modulus $Q = q \cdot p \cdot \Delta^\ell$ for the encryptions S_k , after ℓ rescalings, we end up with an encryption of m under the modulus $q \cdot p > q$. This achieves bootstrapping since there remains a level for a single homomorphic multiplication. For this last level, we can use a smaller scaling factor $p \simeq q^{2/3}$, because the plaintext message m must be encoded under the condition $m = \mathcal{O}(q^{2/3})$ for (7). On the other hand, we must use a scaling factor $\Delta = \mathcal{O}(q)$ for the main bootstrapping evaluation. We prove the following bootstrapping theorem summarizing the above in Appendix E.

Theorem 1 (Bootstrapping). *Given a ciphertext ct such that $\|[\langle \text{ct}, \text{sk} \rangle]_q\|_\infty < q^{2/3}$ and $\langle \text{ct}, \text{sk} \rangle = m + e \pmod{q}$ for $m \in \mathbb{Z}$ and $e \in \mathcal{R}$, the above evaluation procedure with $\Delta \geq \max(4N^2\kappa q, (20N^3\kappa)^2)$ outputs a new ciphertext ct' such that $\langle \text{ct}', \text{sk} \rangle = m + e(0) + e_{\text{bt}} \pmod{p \cdot q}$, where $e(0)$ is the constant coefficient of e , and $\|e_{\text{bt}}\| \leq B_{\text{bt}}$ with $B_{\text{bt}} := 10N$.*

As opposed to the original CKKS bootstrapping, our new bootstrapping equation (8) does not require a small Hamming weight secret key; however, we describe an optimization based on such a secret key in Section 4.3. We note that the above bootstrapping method is fully compatible with RNS arithmetic. In that case, the moduli are of the form $Q_r = \prod_{0 \leq i \leq r} q_i$, for small primes q_i .

4.2 First optimization: packing the secret-key bits

For a ring degree N , the above bootstrapping algorithm requires $\mathcal{O}(N)$ homomorphic operations, which would be impractical. Recall that the CKKS scheme provides a packing method where $N/2$ plaintext slots can be packed into a single ciphertext. These $N/2$ slots can then be added and multiplied independently, by performing homomorphic operations on ciphertexts (see Fig.

3 in Section 3.3). Therefore, the first natural optimization consists of packing the N secret key bits s_k from (8) into the $N/2$ slots of two CKKS ciphertexts, instead of encrypting each bit separately. Similarly, we pack the terms $\exp(2i\pi \cdot c_k/q) - 1$ into two polynomial encodings. The products $(\exp(2i\pi \cdot c_k/q) - 1) \cdot s_k$ are then computed independently and in parallel in the slots, thus requiring only two homomorphic external multiplications (ExtMultR). Moreover, the bootstrapping key size is also reduced from N to only two ciphertexts.

Another advantage of the packing method is that CKKS natively supports complex numbers in the slots, so we do not need to encrypt the real/imaginary part separately anymore. To compute the product in (8), we can homomorphically apply the $\text{Pr}_{N/2 \rightarrow 1}$ operator from Section 3.5 to compute the product of the $N/2$ slots in each ciphertext. As a consequence, in the entire procedure, the multiplicative depth remains $\ell = \log_2(N) + 2$, but now the number of homomorphic multiplications is reduced from $\mathcal{O}(N)$ to $\mathcal{O}(\log N)$. We will describe this optimization more formally in Section 4.4, when combined with the following second optimization.

4.3 Second optimization: using the trace operator

In order to be competitive with the original CKKS bootstrapping, we describe a second optimization. It further reduces the multiplicative depth from $\mathcal{O}(\log N)$ to $\mathcal{O}(\log h)$, with $h \ll N$, by replacing most homomorphic multiplications by additions, thus boosting efficiency.

For this, we still consider a binary secret key s but with a small Hamming weight h , where h is a power-of-two, moreover with a special block structure (as in e.g. [LMSS23]). We assume that the coefficient vector $\mathbf{s} \in \{0, 1\}^N$ can be split into h blocks, each of size $B = N/h$, where each block contains a single 1. Formally, we write $\mathbf{s} = (s_{b \cdot B + j})_{0 \leq b < h, 0 \leq j < B}$ where in the b -th block, $s_{b \cdot B + j} = 0$ for all $j \in \{0, \dots, B-1\} \setminus \{j_b^*\}$, and $s_{b \cdot B + j_b^*} = 1$ for some $0 \leq j_b^* < B$.

As previously, we consider an LWE ciphertext $\mathbf{c} \in \mathbb{Z}_q^N$ derived from the original RLWE ciphertext, with decryption equation:

$$m \simeq \langle \mathbf{s}, \mathbf{c} \rangle = \sum_{k=0}^{N-1} s_k c_k = \sum_{b=0}^{h-1} \sum_{j=0}^{B-1} s_{b \cdot B + j} \cdot c_{b \cdot B + j} \pmod{q}$$

and, again, we embed the above equation into the multiplicative circle group of complex numbers:

$$\exp(2i\pi \cdot \langle \mathbf{s}, \mathbf{c} \rangle / q) = \prod_{b=0}^{h-1} \prod_{j=0}^{B-1} \exp(2i\pi \cdot s_{b \cdot B + j} \cdot c_{b \cdot B + j} / q).$$

Since within a given block there is only a single $s_{b \cdot B + j}$ equal to 1 and the others are zero, we can replace the products within a block by a sum:

$$\exp(2i\pi \cdot \langle \mathbf{s}, \mathbf{c} \rangle / q) = \prod_{b=0}^{h-1} \sum_{j=0}^{B-1} s_{b \cdot B + j} \cdot \exp(2i\pi \cdot c_{b \cdot B + j} / q).$$

As previously, this enables to derive the following bootstrapping equation:

$$\frac{m}{\Delta} \simeq \frac{q}{2\pi\Delta} \cdot \text{Im} \left(\prod_{b=0}^{h-1} \sum_{j=0}^{B-1} s_{b \cdot B + j} \cdot \exp(2i\pi \cdot c_{b \cdot B + j} / q) \right).$$

We can remove the last multiplication by $q/(2\pi\Delta)$ by scaling by a factor $\delta \in \mathbb{R}$ such that $\delta^h = q/(2\pi\Delta)$. This yields our final bootstrapping equation in the single slot case:

$$\frac{m}{\Delta} \simeq \text{Im} \left(\prod_{b=0}^{h-1} \sum_{j=0}^{B-1} s_{b \cdot B + j} \cdot \exp(2i\pi \cdot c_{b \cdot B + j}/q) \cdot \delta \right). \quad (9)$$

The advantage of the above equation is that it has a smaller multiplicative depth than (8), since we are only computing the product of h factors instead of N . More precisely, the multiplicative depth is now $\ell = \log_2(h) + 1$ instead of $\ell = \log_2(N) + 2$. By lowering the multiplicative depth, we can decrease the size of the largest modulus $Q = q \cdot p \cdot \Delta^\ell$. After bootstrapping, the refreshed ciphertext is encrypted under the larger modulus $p \cdot q$ rather than the initial modulus q , enabling a single additional homomorphic multiplication between two bootstrapped ciphertexts. Similar to the original CKKS bootstrapping, further homomorphic operations after bootstrapping can be supported by choosing a larger Q .

4.4 Our bootstrapping algorithm for a single component

We now formally describe our bootstrapping algorithm for a single slot by combining the two previous optimizations. We consider the sparse block secret key $\mathbf{s}$ as in Section 4.3, split into h blocks, each of size $B = N/h$. In order to combine the two optimizations, we must rewrite equation (9) to support packed slots. Recall that we must encode N secret-key bits, but a polynomial over the plaintext space $\mathbb{Z}[X]/(X^N + 1)$ can only encode $N/2$ slots, so we need two polynomial encodings. To speed up computation, we split each block of the secret key in two halves. The first halves of each block will be placed in the first polynomial encoding, and the second halves in the second encoding; see Figure 4 for an illustration.

After the external multiplications by the encoding of the $\exp(2i\pi \cdot c_{b \cdot B + j}/q)$ values, we compute the sum in (9) by first adding the two polynomial encodings, while the rest of the sum's evaluation within each block is handled by the $\text{Tr}_{N/2 \rightarrow h}$ operator applied to a single encoding.

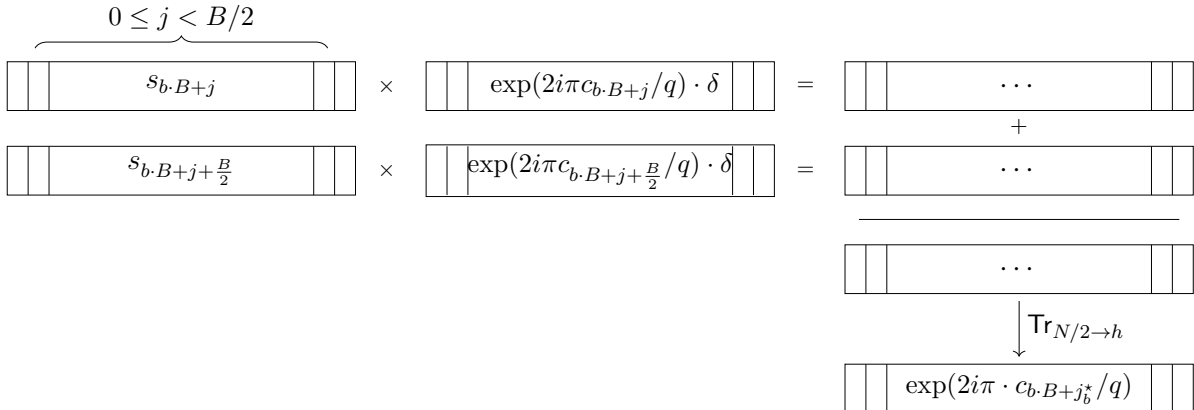


Fig. 4. Illustration of the computation within the half-blocks of size $B/2$. After the application of the $\text{Tr}_{N/2 \rightarrow h}$ operator, all $B/2$ slots contain the same value $\exp(2i\pi \cdot c_{b \cdot B + j_b^*}/q)$, where j_b^* is the index for which $s_{b \cdot B + j_b^*} = 1$. Note that each polynomial encoding contains h such half-blocks, for a total of $h \cdot B/2 = N/2$ slots.

However, we must be careful with the indexing of the slots when applying the $\text{Tr}_{N/2 \rightarrow h}$ operator. As illustrated in Appendix B.1, $\text{Tr}_{N/2 \rightarrow h}$ computes the partial sums of slots sharing the same index modulo h . This implies that we cannot encode the elements of the blocks contiguously in the slots, but only separated modulo h . Therefore, we utilize a “reversed” indexing, in which we put the j -th element of the b -th block at index $j \cdot h + b$ for all $0 \leq j < B/2$ for the first half, and similarly at index $(j - B/2) \cdot h + b$ for all $B/2 \leq j < B$ for the second half. More precisely, we denote by $\tilde{s}_\iota^0$ and $\tilde{s}_\iota^1$ the corresponding slots, for $0 \leq \iota < N/2$. According to the above indexing, we let $\tilde{s}_{j \cdot h + b}^0 = s_{b \cdot B + j}$ for $0 \leq j < B/2$ (first block halves), and $\tilde{s}_{(j - B/2) \cdot h + b}^1 = s_{b \cdot B + j}$ for $B/2 < j < B$ (second block halves), for all $0 \leq b < h$. In total, the N secret-key bits s_k are encoded into the two polynomials:

$$S_0(X) = \text{Ecd}((\tilde{s}_\iota^0)_{0 \leq \iota < N/2}), \quad S_1(X) = \text{Ecd}((\tilde{s}_\iota^1)_{0 \leq \iota < N/2}).$$

We encode the ciphertext’s components following the same indexing. More precisely, we define the slots $(e_\iota^0)_{0 \leq \iota < N/2}$ and $(e_\iota^1)_{0 \leq \iota < N/2}$, with $e_{j \cdot h + b}^0 = \exp(2i\pi \cdot c_{b \cdot B + j}/q) \cdot \delta$ for $0 \leq j < B/2$, and $e_{(j - B/2) \cdot h + b}^1 = \exp(2i\pi \cdot c_{b \cdot B + j}/q) \cdot \delta$ for $B/2 \leq j < B$, for all $0 \leq b < h$. This yields the following polynomial encodings:

$$E_0(X) = \text{Ecd}((e_\iota^0)_{0 \leq \iota < N/2}), \quad E_1(X) = \text{Ecd}((e_\iota^1)_{0 \leq \iota < N/2}).$$

Under this indexing, the bootstrapping equation (9) can be rewritten as:

$$\frac{m}{\Delta} \simeq \text{Im} \left(\prod_{b=0}^{h-1} \sum_{j=0}^{B/2-1} (\tilde{s}_{j \cdot h + b}^0 \cdot e_{j \cdot h + b}^0 + \tilde{s}_{j \cdot h + b}^1 \cdot e_{j \cdot h + b}^1) \right). \quad (10)$$

With this new indexing now compatible with the $\text{Tr}_{N/2 \rightarrow h}$ operator, we can perform the same operation homomorphically on polynomials. Specifically, we use $\text{Tr}_{N/2 \rightarrow h}$ to compute the sums over each half-block and $\text{Pr}_{h \rightarrow 1}$ to calculate the product of the h factors, after which the $N/2$ slots contain the same value. Finally, we extract the imaginary part using the Im2 operator (see Section 3.4). Eventually, from (10), we obtain m/Δ in all $N/2$ slots, which corresponds to an encoding of $m = \text{Ecd}(m/\Delta) \in \mathbb{Z}$. In total, we arrive at our bootstrapping equation for polynomial encodings:

$$m \simeq \text{Im2} \left(\text{Pr}_{h \rightarrow 1} \left(\text{Tr}_{N/2 \rightarrow h} (S_0(X) \times E_0(X) + S_1(X) \times E_1(X)) \right) \right). \quad (11)$$

Note that we must use a scaling factor $\delta \in \mathbb{R}$ such that $\delta^h = q/(4\pi\Delta)$ instead of $\delta^h = q/(2\pi\Delta)$, because of the factor 2 in the Im2 operator.

Bootstrapping. As before, we claim that Equation (11) provides a decryption equation that is compatible with bootstrapping. Namely, all the previous polynomial operations can be applied homomorphically to ciphertexts.

More precisely, let cs denote the bootstrapping key consisting of CKKS encryptions cs_0, cs_1 of $S_0(X)$ and $S_1(X)$, modulo the largest modulus Q . The products $S_i(X) \times E_i(X)$ are then performed homomorphically on ciphertexts using $\text{ExtMultR}(\text{cs}_i, E_i)$, including rescaling. Similarly, the operators $\text{Tr}_{N/2 \rightarrow h}$, $\text{Pr}_{h \rightarrow 1}$ and Im2 are applied homomorphically on ciphertexts, as explained in sections 3.4 and 3.5. Eventually, we obtain an encryption of $m/\Delta \in \mathbb{R}$ over all slots, and equivalently an encryption of $m = \text{Ecd}(m/\Delta) \in \mathbb{Z}$. This corresponds to a new CKKS ciphertext ct' such that $\langle \text{ct}', \text{sk} \rangle = m + e \pmod{pq}$, for a new modulus $pq > q$. For this, it suffices to set the

largest modulus $Q := q \cdot p \cdot \Delta^\ell$, where $\ell = \log_2(h) + 1$ is the multiplicative depth. We provide a pseudo-code description of this bootstrapping algorithm in Appendix F.

The main advantage of the packing approach is that the total number of homomorphic operations is now $\mathcal{O}(\log N)$ instead of $\mathcal{O}(N)$, which makes our bootstrapping equation practical. In terms of the costly homomorphic multiplications, which dominate the number of bit-operations, the complexity further decreases to $\mathcal{O}(\log h)$. We will consider the bootstrapping of $n \geq 2$ slots in parallel in Section 5, and concrete implementation results in Section 6.

4.5 Security analysis of block binary secret key

Our scheme uses a block binary secret key, divided into h blocks, each of size $B = N/h$, with exactly one '1' in each block. This distribution slightly differs from that used in the original CKKS bootstrapping: a ternary secret key with Hamming weight h , but without the regular block structure. To ensure security against the best-known attacks, we provide a specific security analysis for this modified distribution and present corresponding parameter sets in Section 6. We follow the analysis from [LMSS23], which uses the same distribution in the context of TFHE.

We first consider the hybrid dual attack on LWE with a sparse secret [Alb17]. This attack, which combines a dual lattice attack with exhaustive search, is used in the Lattice Estimator [APS15] to estimate the security of LWE. In the first phase, the secret key $\mathbf{s}$ is partitioned as $\mathbf{s} = (\mathbf{s}_0, \mathbf{s}_1)$. Using lattice reduction, the LWE problem is reduced to a single LWE instance with a smaller dimension. The secret key $\mathbf{s}_0$ can then be recovered using combinatorial techniques such as exhaustive search or a meet-in-the-middle algorithm.

For the second, combinatorial phase, the authors of [LMSS23] considered two possible approaches. The original method involves guessing the positions of the zeros in the secret key. However, for the block binary distribution, the lattice dimension can be reduced by the block size $B = N/h$ by guessing the position of a '1' among B possible cases. While the latter approach seemed potentially more efficient, the authors provide an analysis showing that the original approach of guessing the zero positions is, in fact, more efficient. Consequently, the block binary structure seems to offer no advantage for an attacker. Thus, the hybrid dual attack, as modeled in the Lattice Estimator, remains the best attack. Since, for convenience, we assume $s_0 = 1$, we must adjust the ring dimension to $N' = N \cdot (1 - 1/h)$ and the Hamming weight to $h' = h - 1$ in the estimator.

The authors of [LMSS23] also considered the recently improved meet-in-the-middle attack algorithm from May [May21]. It is based on recursively splitting the secret vector $\mathbf{s}$, which requires at least $\mathcal{S}^{0.25}$ time, where $\mathcal{S}$ is the size of the key space. In our case, the search space is $\mathcal{S} = (N/h)^{h-1}$. Therefore, the complexity of May's attack is at least $\mathcal{S} = (N/h)^{0.25(h-1)}$. As in [CHK⁺18a], we will take $h = 64$ in the concrete parameters. In that case, for $N \geq 2^{15}$, the complexity is at least 2^{141} operations.

Finally, we observe that the complexity of our bootstrapping algorithm grows only as $\mathcal{O}(\log h)$ homomorphic multiplications, making it relatively insensitive to increases in h . Even if the attacks exploiting the sparsity of the secret key were to improve significantly, increasing h would not result in a substantial performance penalty. For example, augmenting h from 64 to 128 would only increase the number of homomorphic operations in our bootstrapping by about 7%. Conversely, the complexity of May's attack would escalate from 2^{141} to 2^{254} . This originates from the double-exponential gap between the complexity of our bootstrapping and that of May's attack.

We conclude that, as in the TFHE case, the use of block binary secrets in CKKS does not affect the hardness of RLWE in the usual parameter setup.

5 Bootstrapping for multiple slots

In the previous section, we have described our new CKKS bootstrapping algorithm for a single slot only. That is, we considered a CKKS ciphertext ct such that $\langle \text{ct}, \text{sk} \rangle = m + e \pmod{q}$, for a message $m \in \mathbb{Z}$. In this section, we generalize our bootstrapping equation (9) to handle multiple slots in parallel.

5.1 Bootstrapping equation for multiple slots

For a power-of-two $n \geq 2$, we consider the plaintext space $\mathcal{P}_n = \mathbb{Z}[X^{N/n}]/(X^N+1) \simeq \mathbb{Z}[Y]/(Y^n+1)$ for $Y = X^{N/n}$. Therefore, the plaintext space contains only n coefficients. We consider a ciphertext $\text{ct} = (c_0, c_1)$ encrypting a message $m \in \mathcal{P}_n$.

Our first step is to extract the n corresponding LWE ciphertexts. For this, we consider the decryption equation $\langle \text{ct}, \text{sk} \rangle = c_0 + s \cdot c_1 = m + e \pmod{q}$, where

$$m(X) = \sum_{a=0}^{n-1} m_a X^{aN/n} \in \mathcal{P}_n,$$

and $e \in \mathbb{Z}[X]/(X^N+1)$. For ease of notation, in the decryption equation, we include the coefficients of $X^{aN/n}$ in the error $e(X)$ directly in $m(X)$. By the linearity of polynomial multiplication, we can rewrite the decryption equation as a vector-matrix multiplication, keeping only the coefficients in $X^{aN/n}$ for $0 \leq a < n$. By setting $s(X) = \sum_{i=0}^{N-1} s_i X^i$, the decryption equation can be written as:

$$\sum_{a=0}^{n-1} m_a X^{aN/n} + e = c_0 + c_1 \cdot \sum_{i=0}^{N-1} s_i X^i = c_0 + \sum_{i=0}^{N-1} s_i \cdot c_1 \cdot X^i \pmod{q}.$$

Therefore, we consider the vector $\mathbf{c}_0 \in \mathbb{Z}_q^n$ of coefficients of powers of $X^{N/n}$ of $c_0(X)$, and the matrix $\mathbf{C}_1 \in \mathbb{Z}_q^{N \times n}$ whose i -th row contains the n coefficients of powers of $X^{N/n}$ of $c_1(X) \cdot X^i$, for $0 \leq i < N$. Using this notation, we arrive at the equivalent decryption equation $\mathbf{m} = \mathbf{c}_0 + \mathbf{s} \cdot \mathbf{C}_1 \pmod{q}$. Assuming that $s_0 = 1$, we can add the row vector $\mathbf{c}_0$ to the first row of $\mathbf{C}_1$ and obtain the decryption equation for $\mathbf{m} \in \mathbb{Z}^n$, $\mathbf{s} \in \mathbb{Z}^N$ and $\mathbf{C} \in \mathbb{Z}_q^{N \times n}$:

$$\mathbf{m} = \mathbf{s} \cdot \mathbf{C} \pmod{q} \tag{12}$$

We call $\mathbf{C}$ the *decryption matrix*, and by $\mathbf{C} \leftarrow \text{DecMat}(\text{ct})$ we denote the above algorithm extracting $\mathbf{C} \in \mathbb{Z}_q^{N \times n}$ from the ciphertext ct .

As in Section 4.3, we consider a binary secret key s with a small power-of-two Hamming weight h , such that when written as a coefficient vector $\mathbf{s} \in \{0, 1\}^N$, it can be separated into h blocks, each of size $B = N/h$, where there is exactly a single 1 in each block. From (12), for each component m_a of $\mathbf{m} \in \mathbb{Z}^n$, we can write $m_a = \langle \mathbf{s}, \mathbf{C}_a \rangle \pmod{q}$ for the corresponding column vector $\mathbf{C}_a$ of $\mathbf{C}$. Therefore, each m_a is decrypted as an independent LWE ciphertext given by the column vector $\mathbf{C}_a$. This implies that Equation (9) generalizes to the following bootstrapping equation for each of the m_a for $0 \leq a < n$:

$$\frac{m_a}{\Delta} \simeq \text{Im} \left(\prod_{b=0}^{h-1} \sum_{j=0}^{B-1} s_{b \cdot B + j} \cdot \exp(2i\pi \cdot C_{b \cdot B + j, a}/q) \cdot \delta \right). \tag{13}$$

In the next section, we show how to perform this computation efficiently in parallel using all available $N/2$ slots of polynomials in $\mathbb{Z}[X]/(X^N + 1)$, and then homomorphically over ciphertexts to achieve bootstrapping.

5.2 Bootstrapping algorithm

Our approach for evaluating (13) homomorphically is essentially the same as in Section 4.4. The difference is that we must compute (13) for the n components m_a in parallel instead of a single one. Since each polynomial encoding provides a maximum of $N/2$ slots, we only have $N/(2n)$ slots per component m_a at our disposal in each polynomial encoding. Recall that the binary secret key $\mathbf{s}$ is split into h blocks of size $B = N/h$, each containing a single 1. With only $N/(2n)$ slots at our disposal and h blocks, we will consider sub-blocks of size $N/(2n)/h = B/(2n)$. In other words, we split each block of B components into $2n$ sub-blocks with $B/(2n)$ components each, and we encode each of the $2n$ sub-blocks in a separate polynomial.

$$\begin{array}{c}
 \overbrace{0 \leq k < B/(2n)} \\
 \left\{ \begin{array}{c}
 \boxed{\dots \boxed{s_{b \cdot B + k}} \dots} \times \boxed{\dots \boxed{\exp(2i\pi \cdot C_{b \cdot B + k, a}/q)} \dots} = \boxed{\dots \dots \dots} \\
 \vdots \\
 \boxed{\dots \boxed{s_{b \cdot B + k + (2n-1) \frac{B}{2n}}} \dots} \times \boxed{\dots \boxed{\exp(2i\pi \cdot C_{b \cdot B + k + \cdot, a}/q)} \dots} = \boxed{\dots \dots \dots}
 \end{array} \right. \\
 \hline
 \boxed{\dots \dots \dots} \\
 \downarrow \text{Tr}_{N/2 \rightarrow hn} \\
 \boxed{\dots \boxed{\exp(2i\pi \cdot C_{b \cdot B + j_b^*, a}/q)} \dots}
 \end{array}$$

Fig. 5. Illustration of the computation within a given block. After application of the $\text{Tr}_{N/2 \rightarrow hn}$ operator, all $B/(2n)$ slots contain the same value $\exp(2i\pi \cdot C_{b \cdot B + j_b^*, a}/q)$, where j_b^* is the index for which $s_{b \cdot B + j_b^*} = 1$.

As illustrated in Figure 5, we first compute the products $s_{b \cdot B + j} \cdot \exp(2i\pi \cdot C_{b \cdot B + j, a}/q) \cdot \delta$ from (13) in parallel over each sub-block of size $B/(2n)$, for each of the $2n$ sub-blocks. To compute the sum in (13), we first compute the sum of the $2n$ corresponding encodings, such that we end up with a single sub-block with $B/(2n)$ components. Then we can apply the trace operator to finish the computation of the sum of each block. After the application of the trace operator, each of the $B/(2n)$ slots contains the same value $\exp(2i\pi \cdot C_{b \cdot B + j_b^*, a}/q) \cdot \delta$. This computation is performed in parallel for each of the h blocks and for each of the n components m_a of the plaintext. In total, this corresponds to $h \cdot n$ independent slots; therefore, we must use the $\text{Tr}_{N/2 \rightarrow hn}$ operator. Eventually we apply the $\text{Pr}_{hn \rightarrow n}$ operator to compute the final product of the h elements, for each of the n components of the message in parallel.

As previously, we must use an indexing of the slots that is compatible with the $\text{Tr}_{N/2 \rightarrow hn}$ operator, which computes partial sums of the slots sharing the same index modulo hn (see Appendix B.1 for an illustration). This implies that we must not encode the block's elements contiguously in the slots, but separated modulo nh . Therefore, we use a “reversed” indexing as in Section 4.4. In this reversed indexing, the index $0 \leq j < B$ in (13) is decomposed as

$j = u \cdot B/(2n) + k$ for $0 \leq u < 2n$ and $0 \leq k < B/(2n)$. Consequently, for an index $0 \leq u < 2n$, we consider the slots $(\tilde{s}_\iota^u)_{0 \leq \iota < N/2}$, such that for all $0 \leq k < B/(2n)$, $0 \leq b < h$ and $0 \leq a < n$, we have:

$$\tilde{s}_{k \cdot hn + b \cdot n + a}^u = s_{b \cdot B + u \cdot B/(2n) + k}.$$

We use the same reversed indexing for the ciphertext components, with the slots $(e_\iota^u)_{0 \leq \iota < N/2}$, such that for all $0 \leq k < B/(2n)$, $0 \leq b < h$ and $0 \leq a < n$, we have:

$$e_{k \cdot hn + b \cdot n + a}^u = \exp(2i\pi \cdot C_{b \cdot B + u \cdot B/(2n) + k, a} / q) \cdot \delta.$$

Under this new indexing, we may rewrite the bootstrapping equation (13) as:

$$\frac{m_a}{\Delta} \simeq \text{Im} \left(\prod_{b=0}^{h-1} \sum_{k=0}^{B/(2n)-1} \sum_{u=0}^{2n-1} \tilde{s}_{k \cdot hn + b \cdot n + a}^u \cdot e_{k \cdot hn + b \cdot n + a}^u \right)$$

Since the indexing is now compatible with the $\text{Tr}_{N/2 \rightarrow hn}$ operator, we can transfer the above equation to polynomial notation. Namely, we consider for an index $0 \leq u < 2n$ the polynomials $S_u(X) = \text{Ecd}((\tilde{s}_\iota^u)_{0 \leq \iota < N/2})$ and $E_u(X) = \text{Ecd}((e_\iota^u)_{0 \leq \iota < N/2})$, and we obtain the equivalent equation:

$$\text{Ecd} \left(\left(\frac{m_a}{\Delta} \right)_{0 \leq a < n} \right) \simeq \text{Im}2 \left(\text{Pr}_{hn \rightarrow n} \left(\text{Tr}_{N/2 \rightarrow hn} \left(\sum_{u=0}^{2n-1} S_u(X) \times E_u(X) \right) \right) \right).$$

Note that in the above equation, we only have the decrypted values m_a/Δ in the slots, so we need to move the m_a 's to the coefficient space, using the same **SlotToCoeff** procedure as in the original CKKS bootstrapping procedure (see Section 3.6). This eventually gives us the decrypting equation:

$$\sum_{a=0}^{n-1} m_a X^{aN/n} \simeq \text{StC} \left(\text{Im}2 \left(\text{Pr}_{hn \rightarrow n} \left(\text{Tr}_{N/2 \rightarrow hn} \left(\sum_{u=0}^{2n-1} S_u(X) \times E_u(X) \right) \right) \right) \right).$$

Finally, since **SlotToCoeff** is based on a homomorphic DFT computation to achieve $\mathcal{O}(\log n)$ complexity, the coefficients m_a are, in fact, recovered in bit-reversed order¹. To recover the m_a coefficients with the normal order, we must therefore encode the slots in the polynomials $S_u(X)$ and $E_u(X)$ with bit-reversed order.

Bootstrapping. As previously, we claim that this equation is compatible with bootstrapping, since it can be homomorphically evaluated over ciphertexts, such that eventually, we obtain a refreshed ciphertext of the same plaintext $m(X) = \sum_{a=0}^{n-1} m_a X^{aN/n}$, but under a larger modulus $q \cdot p$.

As in the single slot case, the bootstrapping key consists of CKKS encryptions of the polynomials $S_u(X)$ for $0 \leq u < 2n$; see Alg. 1 below. The products $S_u(X) \times E_u(X)$ are evaluated homomorphically on ciphertexts using the **ExtMultR** procedure.² Similarly, the operators $\text{Tr}_{N/2 \rightarrow hn}$, $\text{Pr}_{hn \rightarrow 1}$ and $\text{Im}2$ are applied homomorphically on ciphertexts; see Alg. 2 below.

¹ In the OpenFHE library, this is also the case for the matrix-vector implementation of **SlotToCoeff**.

² As an optimization, we may compute the rescaling operation inside the **ExtMultR** procedure only after computing the sum over $0 \leq u < 2n$. This way, we only rescale once compared to $2n$ rescalings.

The multiplicative depth is now $\ell = \log_2 h + 1 + \ell_{\text{StC}}$, where ℓ_{StC} is the depth of `SlotToCoeff` (see Appendix D). Therefore, we set the big modulus $Q = \Delta^\ell \cdot q \cdot p$. The total number of homomorphic operations is $\mathcal{O}(n + \log N)$.

Algorithm 1 Bootstrapping key generation, multiple slots

Input: A length N secret key $\mathbf{s}$ with Hamming weight h , and $B = N/h$

Output: A bootstrapping key $\mathbf{cs} = (\mathbf{cs}_u)_{0 \leq u < 2n}$

```

1: for all  $0 \leq u < 2n$  do
2:   for all  $0 \leq k < B/(2n)$  and  $0 \leq b < h$  and  $0 \leq a < n$  do
3:      $\tilde{s}_{k \cdot hn + b \cdot n + a}^u = s_{b \cdot B + u \cdot B/(2n) + k}$ 
4:    $S_u(X) \leftarrow \text{Ecd}((\tilde{s}_\iota^u)_{0 \leq \iota < N/2})$ 
5:    $\mathbf{cs}_u \leftarrow \text{Enc}_{\text{pk}}(S_u(X))$ 
6: return  $(\mathbf{cs}_u)_{0 \leq u < 2n}$ 

```

Algorithm 2 Bootstrapping, for at most $B/2$ slots

Input: A modulus q , a bootstrapping key $(\mathbf{cs}_u)_{0 \leq u < 2n}$, an RLWE ciphertext ct containing $n \leq B/2$ slots, where $B = N/h$, and $\delta = (q/(4\pi\Delta))^{1/h}$

Output: A refreshed ciphertext ct' modulo $p \cdot q$.

```

1:  $\mathbf{C} \leftarrow \text{DecMat}(\text{ct})$ 
2:  $\text{acc} \leftarrow (0, 0)$ 
3: for  $u = 0$  to  $2n - 1$  do
4:   for all  $0 \leq k < B/(2n)$  and  $0 \leq b < h$  and  $0 \leq a < n$  do
5:      $e_{k \cdot hn + b \cdot n + a}^u = \exp(2i\pi \cdot \mathbf{C}_{b \cdot B + u \cdot B/(2n) + k, a} / q) \cdot \delta$ 
6:    $E_u(X) \leftarrow \text{Ecd}((e_\iota^u)_{0 \leq \iota < N/2})$ 
7:    $T_u \leftarrow \text{ExtMultR}(\mathbf{cs}_u, E_u)$ 
8:    $\text{acc} \leftarrow \text{Add}(\text{acc}, T_u)$ 
9: return  $\text{SlotToCoeff}(\text{Im2}(\text{Pr}_{hn \rightarrow n}(\text{Tr}_{N/2 \rightarrow hn}(\text{acc}))))$ 

```

Note that as previously, the above bootstrapping method is fully compatible with RNS arithmetic.

5.3 Bootstrapping up to N slots.

The previous bootstrapping algorithm is limited to bootstrapping up to $n \leq n_{\max} = B/2 = N/(2h)$ components. However, it can be extended to support more slots, specifically $n' \leq N$, by shifting the coefficients of the input ciphertext and applying the previous bootstrapping procedure as a black box for each group of $n_{\max}$ coefficients. We refer to Appendix G for the details. The number of homomorphic operations remains $\mathcal{O}(n + \log N)$ for n slots and the depth also remains unchanged. Note that our bootstrapping method is highly parallelizable; with n processors, it achieves the same $\mathcal{O}(\log N)$ complexity as the original CKKS bootstrapping. We summarize the asymptotic complexities of CKKS bootstrapping approaches in Table 1, expressed in the number of homomorphic operations.

6 Implementation and performance comparison

In this section, we describe the results of an implementation of our new bootstrapping based on the OpenFHE library [BAB⁺22]. We compare it against the state-of-the-art implementation of the original CKKS bootstrapping also available in OpenFHE. The source code is provided in

https://github.com/coron/spru_boot

	Complexity
Original CKKS bootstrapping [CHK ⁺ 18a]	$\mathcal{O}(\log n + \log N)$
Blind rotation bootstrapping [KDE ⁺ 21]	$\mathcal{O}(n \cdot N)$
Our new bootstrapping	$\mathcal{O}(n + \log N)$

Table 1. Complexities of different bootstrapping approaches for CKKS, expressed in number of ciphertext multiplications, for a ring dimension N and n slots.

6.1 Parameter sets

In the OpenFHE library [BAB⁺22], the ring dimension is automatically determined based on the size of the largest modulus and the desired security level, which is 128-bit by default. We also independently verified the security level of our parameter sets using the standard Lattice Estimator [APS15]. The complete parameter sets used for both the original and our new bootstrapping methods are summarized in Table 2.

Fixing the precision. With the RNS arithmetic, the ciphertext moduli have the form $Q_r = \prod_{0 \leq i \leq r} q_i$, where each q_i is a small prime and $0 \leq r \leq \ell_m$, with ℓ_m being the maximum depth. OpenFHE allows to determine the bit-size of q_0 and the bit-size of the q_i 's for $i \geq 1$ separately. For the original CKKS bootstrapping, OpenFHE uses $\log_2 q_0 \simeq 60$ and $\log_2 q_i \simeq 59$ for $i \geq 1$ achieving roughly 14 bits of precision, with a scaling factor $\Delta = 2^{59}$. For our bootstrapping, we use $\log_2 q_0 \simeq 60$ and $\log_2 q_i \simeq 49$ for $i \geq 1$ to reach the same level of precision, with scaling factor $\Delta = 2^{49}$. In Table 2, we denote by $Q = Q_{\ell_m}$ the largest modulus in the ladder, and by PQ the modulus used for key switching.

Bootstrapping depth. We denote by ℓ_c the number of levels consumed by bootstrapping, excluding `CoeffToSlot` and `SlotToCoeff`. Let ℓ_{op} be the number of levels available after bootstrapping. The total depth ℓ_m is then given by $\ell_m = \ell_{\text{CtS}} + \ell_c + \ell_{\text{StC}} + \ell_{op}$, where ℓ_{CtS} and ℓ_{StC} are the number of levels consumed by `CoeffToSlot` and `SlotToCoeff`, respectively.

Since our goal is to minimize bootstrapping latency, we only leave a single level for computation after bootstrapping, that is $\ell_{op} = 1$, for both schemes. The implementation of the original CKKS bootstrapping in OpenFHE has depth $\ell_c = 11$. For our bootstrapping, we use a Hamming weight $h = 64$ (see our analysis in Section 4.5), which gives $\ell_c = \log_2 h + 1 = 7$.

Depth of `CoeffToSlot` and `SlotToCoeff`. In the OpenFHE C++ library, one can directly fix the number of levels ℓ_{CtS} and ℓ_{StC} for `CoeffToSlot` and `SlotToCoeff`. The complexity (in number of homomorphic operations) is then $\mathcal{O}(\rho \cdot n^{1/\rho})$ where ρ is the number of levels and n the number of slots in sparse packing. Therefore, when bootstrapping a small number of slots (up to $n = 16$), it is more advantageous to use the matrix-vector multiplication technique for both `CoeffToSlot` and `SlotToCoeff`, which gives $\ell_{\text{CtS}} = \ell_{\text{StC}} = 1$. When increasing the number of slots, one can allocate a larger level budget, up to $\ell_{\text{CtS}} = \ell_{\text{StC}} = 4$ for full slot ($n = N/2$) bootstrapping. Note that our new bootstrapping does not use `CoeffToSlot`, therefore $\ell_{\text{CtS}} = 0$ for our bootstrapping.

In the following, we consider bootstrapping only for a relatively small number of slots n , with $1 \leq n \leq 1024$. For the original CKKS bootstrapping, we use $\ell_{\text{CtS}} = \ell_{\text{StC}} = 1$ for $1 \leq n \leq 16$, and $\ell_{\text{CtS}} = \ell_{\text{StC}} = 2$ for $32 \leq n \leq 1024$. For our new bootstrapping, internally we use at most 32 slots, following the technique from Section 5.3. Therefore, we can use $\ell_{\text{StC}} = 1$, except for $n = 1$ for

which $\ell_{\text{stc}} = 0$, since no SlotToCoeff is required in that case. We summarize the parameters in Table 2 below. Note that to bootstrap $n \geq 2$ complex slots, we must consider the plaintext space $\mathcal{P}_{2n} = \mathbb{Z}[X^{N/(2n)}]/(X^N + 1)$, which corresponds to polynomials with $2n$ coefficients (see Section 5.1). Due to its smaller multiplicative depth, our new bootstrapping can operate with a smaller ring dimension $N = 2^{15}$, compared to $N = 2^{16}$ for the original CKKS bootstrapping.

	N	$\log q_0$	$\log q_i$	ℓ_c	n	ℓ_{cts}	ℓ_{stc}	ℓ_m	$\log Q$	$\log PQ$	precision
CKKS boot.	2^{16}	60	59	10	$1 \leq n \leq 16$	1	1	13	828	1128	14 bits
					$32 \leq n \leq 1024$	2	2	15	946	1306	
New boot.	2^{15}	60	49	7	$n = 1$	0	0	8	453	633	14 bits
					$2 \leq n \leq 1024$	0	1	9	502	742	

Table 2. Parameter sets for the two CKKS bootstrapping, for 128 bits of security.

6.2 Running time comparison

We benchmark our implementation using the parameters in Table 2, on an Apple M2 CPU with 8 cores. We also run the OpenFHE implementation of the original CKKS bootstrapping for comparison.

Number of slots	1	2	4	8	16	32	64	128	256	512	1024
CKKS boot.	1.7 s	1.7 s	1.7 s	1.7 s	1.8 s	2.0 s	2.2 s	2.8 s	3.1 s	3.6 s	3.6 s
New boot.	0.3 s	0.4 s	0.4 s	0.4 s	0.5 s	0.6 s	1.3 s	2.7 s	5.6 s	11.3 s	23.6 s

Table 3. Running time comparison of the original CKKS bootstrapping and our new bootstrapping, for a variable number of slots n .

Table 3 shows that, due to its reduced multiplicative depth, our bootstrapping is approximately $5\times$ faster than the original CKKS bootstrapping for a single slot ($n = 1$). It remains significantly faster up to $n = 64$ slots. As illustrated in Figure 6, the crossover point lies around $n = 128$ slots, after which the original CKKS bootstrapping scales as $\mathcal{O}(\log n)$, whereas our new bootstrapping scales as $\mathcal{O}(n)$, thus it becomes impractical for $n > 1024$.

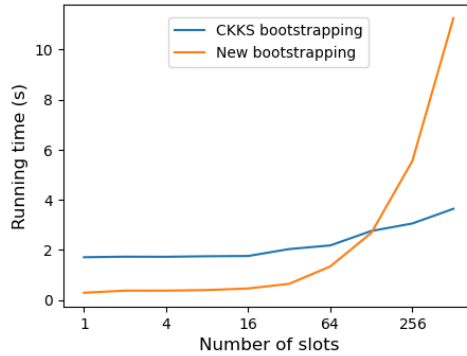


Fig. 6. Comparison of the running times of CKKS bootstrapping and our new bootstrapping.

7 Conclusion

In this paper, we presented an alternative bootstrapping algorithm for the CKKS scheme, based on embedding the additive group modulo q into the circle group of complex numbers, which can be evaluated natively in CKKS. Due to its lower multiplicative depth, our new bootstrapping algorithm can operate with a smaller ring dimension N , resulting in significant efficiency improvements for few slots n . This new bootstrapping approach could be particularly useful in scenarios where circuit evaluation requires a dynamic number of slots, offering an alternative to scheme-switching methods.

References

- Alb17. Martin R. Albrecht. On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 103–129, 2017.
- AP14. Jacob Alperin-Sheriff and Chris Peikert. Faster bootstrapping with polynomial error. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology - CRYPTO 2014 - 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2014, Proceedings, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 297–314. Springer, 2014.
- APS15. Martin Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9, 10 2015.
- BAB⁺22. Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915, 2022.
- BCC⁺22. Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22*, page 223–234, New York, NY, USA, 2022. Association for Computing Machinery.
- BCKS24. Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. Bootstrapping bits with CKKS. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 94–123. Springer, 2024.
- BGV11. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. Fully homomorphic encryption without bootstrapping. *Electron. Colloquium Comput. Complex.*, page 111, 2011.
- Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer, 2012.
- CCKS23. Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23*, page 696–710, New York, NY, USA, 2023. Association for Computing Machinery.
- CCS19. Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 34–54. Springer, 2019. <https://eprint.iacr.org/2018/1043.pdf>.
- CDKS20. Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient homomorphic conversion between (ring) LWE ciphertexts. Cryptology ePrint Archive, Paper 2020/015, 2020. <https://eprint.iacr.org/2020/015>.

- CGGI16. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 3–33, 2016.
- CHH18. Jung Hee Cheon, Kyoohyung Han, and Minki Hhan. Faster homomorphic discrete fourier transforms and improved FHE bootstrapping. *Cryptology ePrint Archive*, Paper 2018/1073, 2018.
- CHK⁺18a. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 360–384. Springer, 2018.
- CHK⁺18b. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full RNS variant of approximate homomorphic encryption. In Carlos Cid and Michael J. Jacobson Jr., editors, *Selected Areas in Cryptography - SAC 2018 - 25th International Conference, Calgary, AB, Canada, August 15-17, 2018, Revised Selected Papers*, volume 11349 of *Lecture Notes in Computer Science*, pages 347–368. Springer, 2018.
- CHKS25. Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. Ship: A shallow and highly parallelizable ckks bootstrapping algorithm. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025*, 2025.
- CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017.
- DM15. Léo Ducas and Daniele Micciancio. FHEW: bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640. Springer, 2015.
- DMPS24. Nir Drucker, Guy Moshkovich, Tomer Pelleg, and Hayim Shaul. BLEACH: cleaning errors in discrete computations over CKKS. *J. Cryptol.*, 37(1):3, 2024.
- FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *IACR Cryptol. ePrint Arch.*, page 144, 2012.
- Gen09. Craig Gentry. Fully homomorphic encryption using ideal lattices. volume 9, pages 169–178, 05 2009.
- GHS12a. Craig Gentry, Shai Halevi, and Nigel P. Smart. Fully homomorphic encryption with polylog overhead, 2012.
- GHS12b. Craig Gentry, Shai Halevi, and Nigel P. Smart. Homomorphic evaluation of the AES circuit. In *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 850–867. Springer, 2012.
- GSW13. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I*, pages 75–92, 2013.
- JKA⁺21. Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Cheon, and Younho Lee. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021:114–148, 08 2021.
- JM22. Charanjit S. Jutla and Nathan Manohar. Sine series approximation of the mod function for bootstrapping of approximate he. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022*, pages 491–520, Cham, 2022. Springer International Publishing.
- KDE⁺21. Andrey Kim, Maxim Deryabin, Jieun Eom, Rakyoung Choi, Yongwoo Lee, Whan Ghang, and Donghoon Yoo. General bootstrapping approach for RLWE-based homomorphic encryption. *Cryptology ePrint Archive*, Paper 2021/691, 2021. <https://eprint.iacr.org/2021/691>.
- KPK⁺23. Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. EvalRound algorithm in CKKS bootstrapping. In *Advances in Cryptology - ASIACRYPT 2022: 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5–9, 2022, Proceedings, Part II*, page 161–187, Berlin, Heidelberg, 2023. Springer-Verlag.

- LLK⁺22. Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022*, pages 551–580, Cham, 2022. Springer International Publishing.
- LLL⁺21. Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In *Advances in Cryptology – EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part I*, page 618–647, Berlin, Heidelberg, 2021. Springer-Verlag.
- LMSS23. Changmin Lee, Seonhong Min, Jinyeong Seo, and Yongsoo Song. Faster TFHE bootstrapping with block binary keys. In Joseph K. Liu, Yang Xiang, Surya Nepal, and Gene Tsudik, editors, *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security, ASIA CCS 2023, Melbourne, VIC, Australia, July 10–14, 2023*, pages 2–13. ACM, 2023.
- LPR10. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.
- May21. Alexander May. How to meet ternary LWE keys. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 701–731. Springer, 2021.
- SSTX09. Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. Efficient public key encryption based on ideal lattices. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6–10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 617–635. Springer, 2009.
- SV14. N. P. Smart and F. Vercauteren. Fully homomorphic SIMD operations. *Designs, Codes and Cryptography*, 71(1):57–81, Apr 2014.
- SYL⁺23. S. Shen, H. Yang, Y. Liu, Z. Liu, and Y. Zhao. CARM: CUDA-accelerated RNS multiplication in word-wise homomorphic encryption schemes for internet of things. *IEEE Transactions on Computers*, 72(07):1999–2010, jul 2023.

A Proof of lemmas 1, 2, 3, 4 and 5

For the following proofs, we will use two facts. For real numbers $r_1, r_2 \in \mathbb{R}$, it holds that $\| [r_1 \pm r_2] \|_\infty \leq \| [r_1] \|_\infty + \| [r_2] \|_\infty + 1$. Moreover, for $r \in \mathbb{R}[X]/(X^N + 1)$ and $s \in \mathbb{Z}[X]/(X^N + 1)$ with hamming weight h and $\|s\|_\infty = 1$, we have that $\|s \cdot [r(X)] - [s \cdot r(X)]\|_\infty \leq h \leq N$, which can be seen by applying the real number case to polynomial multiplication.

A.1 Proof of Lemma 1

Let $\text{pk} = (-a's + e', a')$ be the public key and ct be the ciphertext, i.e. $\text{ct} = \text{Enc}_{\text{pk}}(m) = v \cdot \text{pk} + (m + e_0, e_1) = ((-a's + e')v + m + e_0, a'v + e_1)$, which gives

$$\begin{aligned} \langle \text{ct}, \text{sk} \rangle &= (-a's + e')v + m + e_0 + s(a'v + e_1) \pmod{q_L} \\ &= m + e_0 + e'v + se_1 \pmod{q_L}. \end{aligned}$$

Writing $\langle \text{ct}, \text{sk} \rangle = m + e_{\text{clean}} \pmod{q_L}$, we get for a signed binary secret key:

$$\|e_{\text{clean}}\|_\infty \leq \|e_0\|_\infty + \|e' \cdot v\|_\infty + \|s \cdot e_1\|_\infty \leq \kappa + N \cdot \kappa \cdot 1 + N \cdot 1 \cdot \kappa \leq 3N\kappa.$$

A.2 Proof of Lemma 2

Let $\gamma = q_\ell/q_{\ell'}$. We write $\mathbf{ct} = (b, a)$, which gives $\mathbf{ct}' = (\lfloor b/\gamma \rfloor, \lfloor a/\gamma \rfloor)$. We have $\langle \mathbf{ct}, \mathbf{sk} \rangle = b + s \cdot a + \xi \cdot q_\ell$ for some $\xi \in \mathbb{Z}$, which gives:

$$\langle \mathbf{ct}, \mathbf{sk} \rangle / \gamma = b/\gamma + s \cdot a/\gamma + \xi \cdot q_{\ell'},$$

and therefore we obtain:

$$\begin{aligned} \lfloor \langle \mathbf{ct}, \mathbf{sk} \rangle / \gamma \rfloor &= \lfloor b/\gamma \rfloor + s \cdot \lfloor a/\gamma \rfloor - e_{rs} \pmod{q_{\ell'}} \\ &= \langle \mathbf{ct}', \mathbf{sk} \rangle - e_{rs} \pmod{q_{\ell'}}, \end{aligned}$$

where $e_{rs} = \lfloor b/\gamma \rfloor - b/\gamma + s \cdot (\lfloor a/\gamma \rfloor - a/\gamma) + \langle \mathbf{ct}, \mathbf{sk} \rangle / \gamma - \lfloor \langle \mathbf{ct}, \mathbf{sk} \rangle / \gamma \rfloor$. This implies $\|e_{rs}\|_\infty \leq \|s\|_1 + 1 \leq 2N$, with $\langle \mathbf{ct}', \mathbf{sk} \rangle = \lfloor \langle \mathbf{ct}, \mathbf{sk} \rangle \cdot q_{\ell'} / q_\ell \rfloor + e_{rs} \pmod{q_{\ell'}}$ as required.

A.3 Proof of Lemma 3

We use as an input an evaluation key $\mathbf{evk} = (b', a') = (-a's + e' + Ps^2, a')$ with $\|e'\|_\infty \leq \kappa$ and ciphertexts $\mathbf{ct}_i$ satisfying $\langle \mathbf{ct}_i, \mathbf{sk} \rangle = b_i + s \cdot a_i \pmod{q_\ell}$ for $i \in \{1, 2\}$. We have:

$$\langle \mathbf{ct}_1, \mathbf{sk} \rangle \cdot \langle \mathbf{ct}_2, \mathbf{sk} \rangle = b_1 b_2 + s(b_1 a_2 + b_2 a_1) + s^2 a_1 a_2 \pmod{q_\ell}.$$

Letting $(d_0, d_1, d_2) := (b_1 b_2, b_1 a_2 + b_2 a_1, a_1 a_2)$, we rewrite the above as:

$$\langle \mathbf{ct}_1, \mathbf{sk} \rangle \cdot \langle \mathbf{ct}_2, \mathbf{sk} \rangle = d_0 + s d_1 + s^2 d_2 \pmod{q_\ell}.$$

We consider d_2 over $\mathbb{Z}[X]/(X^N + 1)$, with $\|d_2\|_\infty \leq q_\ell$. By definition, $\mathbf{ct}_{\text{mult}} = (d_0, d_1) + \lfloor P^{-1} d_2 \cdot \mathbf{evk} \rfloor$, where we evaluate $d_2 \cdot \mathbf{evk}$ modulo Pq_L and then divide the latter by P over the reals. This implies, using $q_\ell | q_L$ and some $\gamma_1, \gamma_2 \in \mathbb{Z}$:

$$\begin{aligned} d_2 \cdot \mathbf{evk} &= d_2 \cdot (-a's + e' + Ps^2, a') + Pq_L(\gamma_1, \gamma_2) \quad \text{over } \mathbb{Z}, \text{ and} \\ \lfloor P^{-1} d_2 \cdot \mathbf{evk} \rfloor &= (\lfloor P^{-1} d_2(-a's + e') \rfloor + d_2 s^2, \lfloor P^{-1} a' d_2 \rfloor) \pmod{q_\ell}. \end{aligned}$$

The above now gives:

$$\begin{aligned} \langle \mathbf{ct}_{\text{mult}}, \mathbf{sk} \rangle &= d_0 + d_1 s + \lfloor P^{-1} d_2(-a's + e') \rfloor + d_2 s^2 + s \lfloor P^{-1} a' d_2 \rfloor \pmod{q_\ell} \\ &= \langle \mathbf{ct}_1, \mathbf{sk} \rangle \cdot \langle \mathbf{ct}_2, \mathbf{sk} \rangle + e_{\text{mult}} \pmod{q_\ell}, \end{aligned}$$

and now we can estimate the size of e_{mult} as follows:

$$\begin{aligned} \|e_{\text{mult}}\|_\infty &= \|\lfloor P^{-1} d_2(-a's + e') \rfloor + s \lfloor P^{-1} d_2 a' \rfloor\|_\infty \\ &\leq \|\lfloor P^{-1} d_2(-a's + e') \rfloor + \lfloor s \cdot P^{-1} d_2 a' \rfloor\|_\infty + \|s\|_1 \\ &\leq \|\lfloor s P^{-1} d_2 a' \rfloor + \lfloor P^{-1} d_2 e' \rfloor + \lfloor s P^{-1} d_2 a' \rfloor\|_\infty + \|s\|_1 + 1 \\ &\leq P^{-1} \cdot Nq_\ell \kappa + N + 2. \end{aligned}$$

Finally, if $P \geq Nq_\ell \kappa$, then $\|e_{\text{mult}}\|_\infty \leq 2N$.

A.4 Proof of Lemma 4

Letting $\text{ct}_{\text{mult}} \leftarrow \text{Mult}_{\text{pk}}(\text{ct}_1, \text{ct}_2)$, we obtain by applying Lemma 3:

$$\begin{aligned}\langle \text{ct}_{\text{mult}}, \text{sk} \rangle &= \langle \text{ct}_1, \text{sk} \rangle \cdot \langle \text{ct}_2, \text{sk} \rangle + e_{\text{mult}} \pmod{q_\ell} \\ &= (\text{Ecd}(x_1) + e_1)(\text{Ecd}(x_2) + e_2) + e_{\text{mult}} \pmod{q_\ell} \\ &= \text{Ecd}(x_1)\text{Ecd}(x_2) + e_m + e_{\text{mult}} \pmod{q_\ell},\end{aligned}$$

for $e_m = e_1e_2 + \text{Ecd}(x_1)e_2 + \text{Ecd}(x_2)e_1$, and $\|e_{\text{mult}}\|_\infty \leq 2N$. Letting $\text{ct}_{\text{multR}} \leftarrow \text{RS}(\text{ct}_{\text{mult}})$, applying Lemma 2, and using $q_\ell/q_{\ell-1} = \Delta$, we get:

$$\begin{aligned}\langle \text{ct}_{\text{multR}}, \text{sk} \rangle &= \lfloor \langle \text{ct}_{\text{mult}}, \text{sk} \rangle / \Delta \rfloor + e_{\text{rs}} \pmod{q_{\ell-1}} \\ &= \lfloor (\text{Ecd}(x_1)\text{Ecd}(x_2) + e_m + e_{\text{mult}}) / \Delta \rfloor + e_{\text{rs}} \pmod{q_{\ell-1}},\end{aligned}$$

where $\|e_{\text{rs}}\|_\infty \leq 2N$. By writing $\text{Ecd}(x_1)\text{Ecd}(x_2)/\Delta = \text{Ecd}(x_1x_2) + e_{\text{ecd}}$, we now reformulate the above as:

$$\begin{aligned}\langle \text{ct}_{\text{multR}}, \text{sk} \rangle &= \text{Ecd}(x_1x_2) + \lfloor e_{\text{ecd}} + (e_m + e_{\text{mult}})/\Delta \rfloor + e_{\text{rs}} \pmod{q_{\ell-1}} \\ &= \text{Ecd}(x_1x_2) + e_{\text{multR}} \pmod{q_{\ell-1}}\end{aligned}$$

Therefore, we have:

$$\|e_{\text{multR}}\|_\infty \leq |e_{\text{ecd}}| + \frac{\|e_m\|_\infty}{\Delta} + \frac{\|e_{\text{mult}}\|_\infty}{\Delta} + \|e_{\text{rs}}\|_\infty + 1.$$

We first bound the error e_{ecd} :

$$\begin{aligned}e_{\text{ecd}} &= \frac{1}{\Delta} \cdot (\text{Ecd}(x_1)\text{Ecd}(x_2) - \Delta \cdot \text{Ecd}(x_1x_2)) = \frac{1}{\Delta} \cdot (\lfloor \Delta x_1 \rfloor \lfloor \Delta x_2 \rfloor - \Delta \lfloor \Delta x_1 x_2 \rfloor) \\ &= \frac{1}{\Delta} \cdot ((\Delta x_1 + \varepsilon_1)(\Delta x_2 + \varepsilon_2) - \Delta(\Delta x_1 x_2 + \varepsilon_{12})) \\ &= \frac{1}{\Delta} (\Delta x_1 \varepsilon_2 + \Delta x_2 \varepsilon_1 + \varepsilon_1 \varepsilon_2 - \Delta \varepsilon_{12})\end{aligned}$$

for some $|\varepsilon_1|, |\varepsilon_2|, |\varepsilon_{12}| \leq 1/2$. Using $|x_i| \leq \nu$, this implies

$$|e_{\text{ecd}}| \leq \nu + 1.$$

Recall that $e_m = e_1e_2 + \text{Ecd}(x_1)e_2 + \text{Ecd}(x_2)e_1$. Using $\text{Ecd}(x_i) \in \mathbb{Z}$ and $|\text{Ecd}(x_i)| = |\lfloor x_i \cdot \Delta \rfloor| \leq \Delta\nu + 1$, this gives:

$$\|e_m\|_\infty \leq NE^2 + 2E \cdot (\Delta\nu + 1) \leq 2NE^2 + 2\nu E\Delta.$$

Now, for e_{multR} , we have:

$$\|e_{\text{multR}}\|_\infty \leq \nu + 1 + \frac{2NE^2}{\Delta} + 2\nu E + \frac{2N}{\Delta} + 2N + 1.$$

Assuming $E^2 \leq \Delta$ and $\nu < N$, we eventually get:

$$\|e_{\text{multR}}\|_\infty \leq 2\nu E + 8N.$$

A.5 Proof of Lemma 5

The proof is similar to the proof of Lemma 3. We input the key-switching key $\text{swk} = (b', a')$, for which $b' = -a's + e' + Ps' \pmod{Pq_L}$. Let $\text{ct} = (b, a)$ be the input ciphertext under $\text{sk}' = (1, s')$. We consider a over $\mathbb{Z}[X]/(X^N + 1)$ with $\|a\|_\infty \leq q_\ell$. The output ciphertext is defined as $\text{ct}' = (b, 0) + \lfloor P^{-1}a \cdot \text{swk} \rfloor \pmod{q_\ell}$, where we evaluate $a \cdot \text{swk}$ modulo Pq_L and then divide by P over the reals. As in the proof of Lemma 3, this gives:

$$\lfloor P^{-1}a \cdot \text{swk} \rfloor = (\lfloor P^{-1}a(-a's + e') \rfloor + as', \lfloor P^{-1}a'a \rfloor) \pmod{q_\ell}.$$

The decryption equation of ct' now indeed satisfies:

$$\begin{aligned} \langle \text{ct}', \text{sk} \rangle &= b + \lfloor P^{-1}a(-a's + e') \rfloor + as' + s \cdot \lfloor P^{-1}a'a \rfloor \pmod{q_\ell} \\ &= \langle \text{ct}, \text{sk}' \rangle + \lfloor P^{-1}a(-a's + e') \rfloor + s \cdot \lfloor P^{-1}a'a \rfloor \pmod{q_\ell} \\ &= \langle \text{ct}, \text{sk}' \rangle + e_{\text{ks}} \pmod{q_\ell}. \end{aligned}$$

Finally, we estimate the size of e_{ks} as follows:

$$\begin{aligned} \|e_{\text{ks}}\|_\infty &= \|\lfloor P^{-1}a(-a's + e') \rfloor + s \cdot \lfloor P^{-1}a'a \rfloor\|_\infty \\ &\leq \|\lfloor P^{-1}a(-a's + e') \rfloor + \lfloor sP^{-1}a'a \rfloor\|_\infty + \|s\|_1 \\ &\leq \|\lfloor P^{-1}aa's \rfloor + \lfloor P^{-1}ae' \rfloor + \lfloor sP^{-1}a'a \rfloor\|_\infty + \|s\|_1 + 1 \\ &\leq N + 2 + \|P^{-1}a \cdot e'\|_\infty \leq N + 2 + P^{-1} \cdot Nq_\ell\kappa \leq N + 3 \leq 2N. \end{aligned}$$

B The trace and product operators

B.1 The trace operator

For a power-of-two a , we define the operator $\text{Tr}_{N/2 \rightarrow a}$ as:

$$\text{Tr}_{N/2 \rightarrow a} = (\text{id} + \Psi_a) \circ (\text{id} + \Psi_{2a}) \circ (\text{id} + \Psi_{4a}) \circ \dots \circ (\text{id} + \Psi_{N/4}).$$

One can show recursively that the $\text{Tr}_{N/2 \rightarrow a}$ operator computes partial sums of the slots sharing the same index modulo a , namely for any $\mathbf{z} \in \mathbb{C}^{N/2}$:

$$\text{Tr}_{N/2 \rightarrow a}(\text{iDFT}(z_0, \dots, z_{N/2-1})) = \text{iDFT}(\underbrace{\mathbf{t}, \dots, \mathbf{t}}_{N/(2a) \text{ rep.}})$$

where $\mathbf{t} = (t_k)_{0 \leq k < a}$, with $t_k = z_k + z_{k+a} + \dots + z_{k+(N/(2a)-1) \cdot a}$. Therefore, the $\text{Tr}_{N/2 \rightarrow a}$ operator outputs an encoding of the a slots $t_0, \dots, t_{a-1}$, which are repeated $N/(2a)$ times to get $N/2$ slots; see Fig. 7 for an illustration.

Namely, the operator $\text{Tr}_{N/2 \rightarrow N/4} = \text{id} + \Psi_{N/4}$ computes the sum of pairs of slots sharing the same index modulo $N/4$. Assuming the property for $\text{Tr}_{N/2 \rightarrow a}$ and using $\text{Tr}_{N/2 \rightarrow a/2} = (\text{id} + \Psi_{a/2}) \circ \text{Tr}_{N/2 \rightarrow a}$, the operator $\text{Tr}_{N/2 \rightarrow a/2}$ computes the sum of slots with the same index modulo $a/2$.

As described in Alg. 3 below, the $\text{Tr}_{N/2 \rightarrow a}$ operator can be homomorphically evaluated, requiring $\log_2(N/(2a))$ automorphisms and key switchings, while consuming no multiplicative levels.

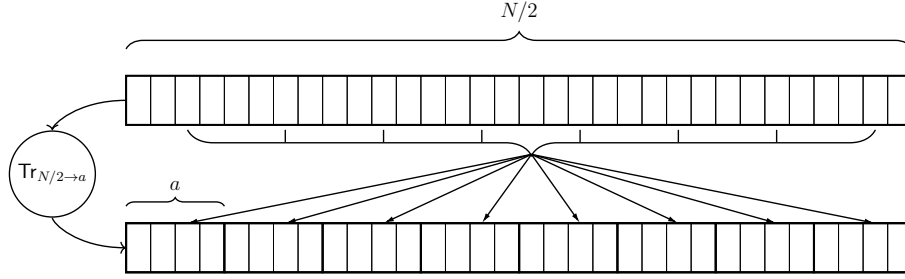


Fig. 7. Illustration of the $\text{Tr}_{N/2 \rightarrow a}$ operator with $N/2 = 32$ slots and $a = 4$. Each slot in the output polynomial is the sum of all input slots with the same index modulo a . Therefore, the output polynomial contains identical blocks of a slots each.

Algorithm 3 Homomorphic trace operator

Input: A ciphertext ct , a power-of-two parameter a

Output: Computes $\text{Tr}_{N/2 \rightarrow a}(\text{ct})$

```

1:  $r = N/4$ 
2: while  $r \geq a$  do
3:    $\text{ct} = \text{Add}(\text{ct}, \text{Rot}(\text{ct}, r))$ 
4:    $r = r/2$ 
5: return  $\text{ct}$ 

```

B.2 The product operator

Algorithm 4 Homomorphic product operator

Input: A ciphertext ct , two power-of-two parameters $a > b$

Output: Computes $\text{Pr}_{a \rightarrow b}(\text{ct})$

```

1:  $r = a/2$ 
2: while  $r \geq b$  do
3:    $\text{ct} = \text{Mult}_{\text{Revk}}(\text{ct}, \text{Rot}(\text{ct}, r))$ 
4:    $r = r/2$ 
5: return  $\text{ct}$ 

```

C The original CKKS bootstrapping procedure

Bootstrapping a single coefficient. We recall the original CKKS bootstrapping approach [CHK⁺18a]. For simplicity, we first consider the bootstrapping of a single coefficient, that is a CKKS ciphertext ct encrypting a message $m \in \mathbb{Z}$, with $\langle \text{ct}, \text{sk} \rangle = m + e \pmod{q}$, where q is the smallest modulus in the ladder.

We first perform a modulus lifting to a larger modulus Q , which gives:

$$\langle \text{ct}, \text{sk} \rangle = qI + m + e \pmod{Q}$$

for some $I \in \mathbb{Z}[X]/(X^N + 1)$. The goal of bootstrapping is to get rid of the qI term. Using the $\text{Tr}_{N/2 \rightarrow 1}$ operator and also $\text{Re2} = \text{id} + \text{conj}$, we have $\text{Re2}(\text{Tr}_{N/2 \rightarrow 1}(qI + m)) = qN \cdot \tilde{I} + N \cdot m$ where $\tilde{I} \in \mathbb{Z}$. Therefore, by applying these operators homomorphically, we can obtain a new ciphertext ct' such that

$$\langle \text{ct}', \text{sk} \rangle = qN \cdot \tilde{I} + Nm + e' \pmod{Q}$$

for a small error e' , so we must remove the term $qN \cdot \tilde{I}$, where now $\tilde{I} \in \mathbb{Z}$ instead of $I \in \mathbb{Z}[X]/(X^N + 1)$.

Such a ciphertext ct' corresponds to an encoding of $x = (qN \cdot \tilde{I} + Nm)/\Delta \in \mathbb{R}$. Eventually, we want to obtain an encryption of m , which corresponds to an encoding of $m/\Delta \in \mathbb{R}$. Therefore, we homomorphically evaluate on ct' a polynomial $P(t)$ such that $m/\Delta \simeq P(x)$. This yields the condition $m/\Delta \simeq P((qN \cdot \tilde{I} + Nm)/\Delta)$, which must hold for any small $\tilde{I} \in \mathbb{Z}$. We expect the integer $\tilde{I}$ to be small because the secret key sk has a small Hamming weight.

For $P(t)$, we can therefore use a polynomial approximation of a periodic function with period qN/Δ , with the appropriate scaling factor:

$$P(t) \simeq \frac{q}{2\pi\Delta} \sin\left(\frac{2\pi\Delta \cdot t}{qN}\right).$$

One could use a Taylor series approximation of the sine function, but the required degree would be too large. Instead, we work over the complex numbers and first consider a degree- d Taylor series approximation of the complex exponential function, with a scaling factor 2^r as input:

$$Q(t) = \sum_{k=0}^d \frac{1}{k!} \left(\frac{2i\pi\Delta \cdot t}{qN2^r}\right)^k \simeq \exp\left(\frac{2i\pi\Delta \cdot t}{qN2^r}\right).$$

Due to the scaling factor 2^r , much fewer coefficients are needed in the Taylor series approximation. Note that $Q(t)$ is a polynomial with complex coefficients, so when homomorphically evaluating $Q(t)$ on ct' , we consider the encryptions of the real part and the imaginary part separately. Finally, we can write

$$P(t) = \frac{q}{2\pi\Delta} \cdot \text{Im}\left(Q(t)^{2^r}\right), \quad (14)$$

which we evaluate homomorphically by performing r successive squarings. Eventually, we only keep the encryption of the imaginary part and apply the scaling factor. Finally, we obtain a new ciphertext ct'' such that

$$\begin{aligned} \langle \text{ct}'', \text{sk} \rangle &\simeq \text{Ecd}(P(x)) \simeq \text{Ecd}(m/\Delta) \pmod{q'} \\ &= m + e'' \pmod{q'} \end{aligned}$$

for a small error e'' . Therefore, we have obtained a new encryption of m for the modulus q' . The previous evaluation procedure has a multiplicative depth of $\ell = d + r + 1$. Therefore, starting from the largest modulus $Q = q \cdot p \cdot \Delta^\ell$, after ℓ rescalings, we end up with a modulus $q' = q \cdot p$. This enables to perform one more homomorphic multiplication on ct' , thereby achieving bootstrapping. We can reduce the depth to $\ell = d + r$ by using a scaling factor δ in $Q(t)$ such that $\delta^{2^r} = q/(2\pi\Delta)$.

Moreover, the depth required to evaluate a polynomial of degree d can be reduced to $\mathcal{O}(\log d)$ at the expense of additional multiplications [CKKS17]. For our parameters, where $d = 7$ is relatively small, this approach slightly reduces the largest modulus Q . However, in practice, we did not observe a significant improvement in running time.

Bootstrapping multiple components. The bootstrapping of multiple components proceeds similarly, which we describe briefly below. For a power-of-two n with $2 \leq n \leq N/2$, we consider the plaintext space $\mathbb{Z}[X^{N/n}]/(X^N + 1) \simeq \mathbb{Z}[Y]/(Y^n + 1)$ for $Y = X^{N/n}$; therefore, the plaintext space has n non-trivial coefficients. Thus, we start from a ciphertext ct with $\langle \text{ct}, \text{sk} \rangle = m + e \pmod{q}$ for $m \in \mathbb{Z}[X^{N/n}]/(X^N + 1)$.

1. Modulus raising: As previously, we consider the same ct modulo a larger modulus Q , which gives $\langle \text{ct}, \text{sk} \rangle = qI + m + e \pmod{Q}$ for a small $I \in \mathbb{Z}[X]/(X^N + 1)$, where $m(X) = \sum_{i=0}^{n-1} m_i \cdot X^{i \cdot N/n}$.
2. Error packing: We homomorphically apply the $\text{Tr}_{N/2 \rightarrow n/2}$ operator, which yields a new ciphertext ct_1 , for which $\langle \text{ct}_1, \text{sk} \rangle = q\tilde{I} \cdot N/n + m \cdot N/n + e \pmod{Q}$, where $\tilde{I} \in \mathbb{Z}[X^{N/n}]/(X^N + 1)$.
3. Coefficients to slots: We use the **CoeffToSlot** procedure recalled previously to homomorphically compute a new ciphertext ct_2 , such that $\langle \text{ct}_2, \text{sk} \rangle = \text{Ecd}(m_0 + q\tilde{I}_0, \dots, m_{n-1} + q\tilde{I}_{n-1}) + e_2 \pmod{Q_2}$.
4. Approximate modular reduction: The polynomial approximation of the reduction function modulo q gets independently applied on each of the n slots, which then yields a new ciphertext ct_3 , for which $\langle \text{ct}_3, \text{sk} \rangle = \text{Ecd}(m_0, \dots, m_{n-1}) + e_3 \pmod{Q_3}$.
5. Slots to coefficients: We use the **SlotToCoeff** procedure recalled previously to homomorphically compute a new ciphertext ct_4 , such that $\langle \text{ct}_4, \text{sk} \rangle = m(X) + e_4 \pmod{q \cdot p}$, for a modulus $q \cdot p$, which is larger than the initial modulus q .

Since the complexity of **CoeffToSlot** and **SlotToCoeff** is $\mathcal{O}(\log n)$ for n slots (using the homomorphic Fast Fourier Transform; see Appendix D), the complexity of the original CKKS bootstrapping for n slots is also $\mathcal{O}(\log n)$.

D Fast DFTs and the **SlotToCoeff** operation

D.1 The DFT and iDFT isomorphisms

Recall that the M -th cyclotomic polynomial is defined as:

$$\Phi_M(x) = \prod_{\substack{1 \leq j \leq M \\ \gcd(j, M)=1}} (x - e^{2\pi j/M})$$

For a power-of-two M , we have $\Phi_M(x) = x^N + 1$ where $N = M/2$. For a power-of-two N , the integer 5 has order $N/2$ modulo $2N$, and generates $\mathbb{Z}_{2N}^*$ together with the integer $\{-1\}$. Let $\zeta = e^{I\pi/N}$ be a primitive $2N$ -th root of unity. Then letting $\zeta_j := \zeta^{5^j}$ for $0 \leq j < N/2$, the set $\{\zeta_j, \overline{\zeta_j} : 0 \leq j < N/2\}$ forms the set of the primitive $2N$ -th roots of unity.

The variant of the Fourier transform DFT_N over $\mathcal{S}_N$ considered in [CKKS17] is defined as

$$\begin{aligned} \text{DFT}_N : \mathcal{S}_N &\rightarrow \mathbb{C}^{N/2} \\ p &\mapsto \left(p(\zeta^{5^0}), p(\zeta^{5^1}), \dots, p(\zeta^{5^{N/2-1}}) \right). \end{aligned}$$

We denote by $\text{iDFT}_N : \mathbb{C}^{N/2} \rightarrow \mathcal{S}_N$ the inverse transform.

The DFT_N function is a ring homomorphism from the ring $\mathcal{S}_N$ (the coefficients space) to the ring of complex vectors in $\mathbb{C}^{N/2}$ (the slots space). Its purpose is therefore to map polynomial addition and multiplication in the ring $\mathcal{S}_N$ to component-wise addition and multiplication in $\mathbb{C}^{N/2}$.

D.2 DFT and iDFT on sub-rings

We consider any divisor n of N with $1 \leq n \leq N/2$, and a polynomial p in the subring $\mathbb{R}[X^{N/n}]/(X^N + 1)$. We can write $p(X) = \tilde{p}(Y) \in \mathbb{R}[Y]/(Y^n + 1)$ where $Y = X^{N/n}$. Then

$$\text{DFT}_N(p) = \left(p(\zeta^{5^i}) \right)_{0 \leq i < N/2} = \left(\tilde{p}(\zeta^{5^i \cdot N/n}) \right)_{0 \leq i < N/2}$$

Since 5 has order $n/2$ modulo $2n$, $\text{DFT}_N(p)$ is periodic with period $n/2$, with N/n repetitions of the same pattern:

$$\text{DFT}_N(p) = \underbrace{(\text{DFT}_n(\tilde{p}), \dots, \text{DFT}_n(\tilde{p}))}_{N/n \text{ rep.}}$$

Conversely, for any $\mathbf{z} \in \mathbb{C}^{n/2}$, we have:

$$\text{iDFT}_N(\underbrace{(\mathbf{z}, \dots, \mathbf{z})}_{N/n \text{ rep.}}) = \text{iDFT}_n(\mathbf{z})(X^{N/n})$$

D.3 Computing DFT and iDFT

Let $m(X) = \sum_{i=0}^{N-1} m_i X^i \in \mathbb{R}[X]/(X^N + 1)$ and let $\mathbf{m} = (m_0, \dots, m_{N-1}) \in \mathbb{R}^N$ be the vector of its coefficients. Letting $\zeta = e^{I\pi/N}$ and $\zeta_j := \zeta^{5^j}$ for $0 \leq j < N/2$ with $\zeta = e^{I\pi/N}$, we have $\mathbf{z} = \text{DFT}_N(\mathbf{m}) = U \cdot \mathbf{m} \in \mathbb{C}^{N/2}$ where

$$U = \begin{bmatrix} 1 & \zeta_0 & \dots & \zeta_0^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_{N/2-1} & \dots & \zeta_{N/2-1}^{N-1} \end{bmatrix}$$

is the $(N/2) \times N$ Vandermonde matrix generated by $\{\zeta_j : 0 \leq j < N/2\}$. The decoding algorithm DFT is therefore a linear transformation from $\mathbb{R}^N$ to $\mathbb{C}^{N/2}$, given by the matrix U .

We consider the full Vandermonde matrix CRT generated by the set $\{\zeta_j, \bar{\zeta}_j : 0 \leq j < N/2\}$, which gives $\text{CRT} = (U; \bar{U})$, and $(\mathbf{z}, \bar{\mathbf{z}}) = (U\mathbf{m}; \bar{U}\mathbf{m}) = \text{CRT} \cdot \mathbf{m}$. We can then compute $\mathbf{m} = \text{CRT}^{-1}(\mathbf{z}, \bar{\mathbf{z}})$, where $\text{CRT}^{-1} = \frac{1}{N} \overline{\text{CRT}}^T$. This enables to write the coefficients $\mathbf{m}$ of the polynomial $m(X) = \text{iDFT}_N(\mathbf{z})$ as the linear transformation from $\mathbb{C}^{N/2}$ to $\mathbb{R}^N$:

$$\mathbf{m} = \frac{1}{N} \left(\bar{U}^T \cdot \mathbf{z} + U^T \cdot \bar{\mathbf{z}} \right)$$

D.4 Fast computation of the DFT and bit-reversed order

The drawback of the technique from the previous section to compute the DFT is that its complexity is $\mathcal{O}(N^2)$ operations. In this section, we recall the fast computation of the DFT function, with complexity $\mathcal{O}(N \log N)$.

We work in the ring $\mathcal{S} = \mathbb{R}[X]/(X^N + 1)$. Using $X^N + 1 = (X^{N/2} - I)(X^{N/2} + I)$, we have the isomorphism:

$$\begin{aligned} \Phi : \mathbb{R}[X]/(X^N + 1) &\simeq \mathbb{C}[X]/(X^{N/2} - I) \\ a &\mapsto (a' = a \bmod (X^{N/2} - I)). \end{aligned}$$

The coefficients $a'_j \in \mathbb{C}$ of a' can be computed as $a'_j = a_j + I \cdot a_{j+N/2}$. Conversely, we have $a_j = (a'_j + \bar{a}'_j)/2$ and $a_{j+N/2} = (a'_j - \bar{a}'_j)/(2I)$. Therefore, the real part of the complex coefficients of a' originates from the first $N/2$ coefficients of $a \in \mathcal{S}$, while the imaginary part comes from the last $N/2$ coefficients of a .

We now consider the ring $\mathbb{C}[X]/(X^{N/2} - I)$. We use $\zeta = e^{I\pi/N}$. Using $\zeta^{N/4} = e^{I\pi/4}$ and $\zeta^{5N/4} = e^{5I\pi/4} = -\zeta^{N/4}$, we can further split the ring as follows:

$$\mathbb{C}[X]/(X^{N/2} - I) \simeq \mathbb{C}[X]/(X^{N/4} - \zeta^{N/4 \cdot (5^0 \bmod 8)}) \times \mathbb{C}[X]/(X^{N/4} - \zeta^{N/4 \cdot (5^1 \bmod 8)}).$$

We can generalize this approach as follows. Fix any integer $k \geq 1$ such that $2^k < N$. For any $0 \leq j < 2^{k-1}$, consider the root $\zeta^{N/2^{k+1} \cdot 5^j}$. It has two square roots $\zeta^{N/2^{k+1} \cdot 5^j}$ and $-\zeta^{N/2^{k+1} \cdot 5^j}$. Moreover, we have $5^{2^{k-1}} = 1 + 2^{k+1} \pmod{2^{k+2}}$. This gives, for any $0 \leq j < 2^{k-1}$:

$$5^{j+2^{k-1}} = 5^j \cdot (1 + 2^{k+1}) = 5^j + 2^{k+1} \pmod{2^{k+2}}.$$

This enables to write the two square roots as $\zeta^{N/2^{k+1} \cdot 5^j}$ and $\zeta^{N/2^{k+1} \cdot 5^{j+2^{k-1}}}$, which yields the isomorphism for $0 \leq j < 2^{k-1}$:

$$\begin{aligned} \mathbb{C}[X]/(X^{N/2^k} - \zeta^{N/2^k \cdot 5^j}) &\simeq \mathbb{C}[X]/(X^{N/2^{k+1}} - \zeta^{N/2^{k+1} \cdot 5^j}) \\ &\quad \times \mathbb{C}[X]/(X^{N/2^{k+1}} - \zeta^{N/2^{k+1} \cdot 5^{j+2^{k-1}}}). \end{aligned}$$

For the corresponding isomorphism with $u = \zeta^{N/2^{k+1} \cdot 5^j}$

$$\Phi_k(a) = \left(a' = a \bmod (X^{N/2^{k+1}} - u), \quad a'' = a \bmod (X^{N/2^{k+1}} + u) \right),$$

The coefficients of a' and a'' can be computed efficiently via

$$\begin{aligned} a'_i &= a_i + u \cdot a_{i+m} \\ a''_i &= a_i - u \cdot a_{i+m} \end{aligned}$$

for $0 \leq i < m$, for $m = N/2^{k+1}$. This is known as the Cooley-Tukey butterfly.

We can then apply this technique recursively. After step k for $1 \leq k \leq \log_2(N/2)$, we arrive at the isomorphism:

$$\mathbb{C}[X]/(X^{N/2} - I) \simeq \prod_{i=0}^{2^k-1} \mathbb{C}[X]/(X^{N/2^{k+1}} - \zeta^{N/2^{k+1} \cdot 5^{\text{br}_k(i)}}).$$

where $\text{br}_k(i)$ is the bit-reversal of the k -bit integer i . It has the properties $\text{br}_k(2i) = \text{br}_{k-1}(i)$ and $\text{br}_k(2i+1) = 2^{k-1} + \text{br}_{k-1}(i)$.

Eventually, for $N = 2^\ell$, we obtain the evaluation of the input polynomial at the roots $\zeta^{5^{\text{br}_{\ell-1}(i)}}$ for $0 \leq i < N/2$, therefore in bit-reversed order. Recall that we have defined $\text{DFT}_N(p)$ as $\left(p(\zeta^{5^i}) \right)_{0 \leq i < N/2}$. Let Br_k denote the function taking as input a sequence of 2^k integers and outputting it in bit-reversed order. With this notation, the above procedure yields a sequence $(v_i)_{0 \leq i < N/2} = \left(p(\zeta^{5^{\text{br}_{\ell-1}(i)}}) \right)_{0 \leq i < N/2} = \text{Br}_{\ell-1}(\text{DFT}_N(p))$. In total, we obtain the following fast DFT_N algorithm, from normal order (no) to bit-reversed order (bo).

Algorithm 5 Fast computation of DFT_N , no $\rightarrow$ bo

Input: A polynomial $p(X) \in \mathbb{R}[X]/(X^N + 1)$, with $p(X) = \sum_{j=0}^{N-1} p_j X^j$, $\zeta = \exp(I\pi/N)$ and $N = 2^\ell$.

Output: A sequence $(v_i)_{0 \leq i < N/2} = \text{Br}_{\ell-1}(\text{DFT}_N(p)) = \left(p(\zeta^{5^{\text{br}_{\ell-1}(i)}}) \right)_{0 \leq i < N/2}$.

```
1:  $(v_j)_{0 \leq j < N/2} = (p_j + I \cdot p_{j+N/2})_{0 \leq j < N/2}$ 
2: for  $k = 1$  to  $\ell - 1$  do
3:   for  $j = 0$  to  $2^{k-1} - 1$  do
4:      $u = \zeta^{5^{\text{br}_{k-1}(j)} \cdot N/2^{k+1}}$ 
5:     for  $i = 0$  to  $N/2^{k+1} - 1$  do
6:        $a, b \leftarrow v_{j \cdot N/2^k + i}, v_{j \cdot N/2^k + i + N/2^{k+1}}$ 
7:        $v_{j \cdot N/2^k + i} = a + u \cdot b$ 
8:        $v_{j \cdot N/2^k + i + N/2^{k+1}} = a - u \cdot b$ 
9: return  $(v_i)_{0 \leq i < N/2}$ 
```

We describe the corresponding iDFT algorithm, that computes the same operations in reverse order.

Algorithm 6 Fast computation of iDFT_N , bo $\rightarrow$ no

Input: A sequence $(v_i)_{0 \leq i < N/2}$, $\zeta = \exp(I\pi/N)$ and $N = 2^\ell$.

Output: A polynomial $p(x) \in \mathbb{R}[X]/(X^N + 1)$, such that $(v_i)_{0 \leq i < N/2} = \text{Br}_{\ell-1}(\text{DFT}_N(p))$

```
1: for  $k = \ell - 1$  downto  $1$  do
2:   for  $j = 0$  to  $2^{k-1} - 1$  do
3:      $u = \zeta^{5^{\text{br}_{k-1}(j)} \cdot N/2^{k+1}}$ 
4:     for  $i = 0$  to  $N/2^{k+1} - 1$  do
5:        $a, b \leftarrow v_{j \cdot N/2^k + i}, v_{j \cdot N/2^k + i + N/2^{k+1}}$ 
6:        $v_{j \cdot N/2^k + i} = (a + b)/2$ 
7:        $v_{j \cdot N/2^k + i + N/2^{k+1}} = u^{-1} \cdot (a - b)/2$ 
8: return  $p(x) = \sum_{i=0}^{N/2-1} \text{Re}(v_i)x^i + \text{Im}(v_i)x^{i+N/2}$ 
```

In Algorithm 7, we describe the same DFT_N algorithm as in Alg. 5, but with the input coefficients of the polynomial in bit-reversed order, and the output coefficients in normal order. In fact, we use a modified bit-reversed indexing, in which the first and second half of the coefficients are encoded separately in bit-reversed order. This will later facilitate the homomorphic evaluation of the algorithm.

Due to this modified bit-reversed encoding, Line 1 of Alg. 5 remains the same. At Line 3, we can equivalently run j' from 0 to $2^{k-1} - 1$, and let $j = \text{br}_{k-1}(j')$. At lines 6, 7, 8, we have

$$\begin{aligned} \text{br}_{\ell-1}(j \cdot N/2^k + i) &= \text{br}_{\ell-k-1}(i) \cdot 2^k + \text{br}_{k-1}(j) \\ \text{br}_{\ell-1}(j \cdot N/2^k + i + N/2^{k+1}) &= \text{br}_{\ell-k-1}(i) \cdot 2^k + 2^{k-1} + \text{br}_{k-1}(j) \end{aligned}$$

This gives the following algorithm.

Algorithm 7 Fast computation of DFT_N , $\text{bo}' \rightarrow \text{no}$

Input: A sequence $(p_i)_{0 \leq i < N/2} \in \mathbb{R}^{N/2}$, $\zeta = \exp(I\pi/N)$ and $N = 2^\ell$.

Output: A sequence $(v_i)_{0 \leq i < N/2} = \text{DFT}(p) = \left(p(\zeta^{5^i}) \right)_{0 \leq i < N/2}$, where $p(X) = \sum_{j=0}^{N/2-1} p_{\text{br}_{\ell-1}(j)} X^j +$

$$\sum_{j=0}^{N/2-1} p_{N/2+\text{br}_{\ell-1}(j)} X^{N/2+j}.$$

1: $(v_j)_{0 \leq j < N/2} = (p_j + I \cdot p_{j+N/2})_{0 \leq j < N/2}$

2: **for** $k = 1$ to $\ell - 1$ **do**

3: **for** $j = 0$ to $2^{k-1} - 1$ **do**

4: $u = \zeta^{5^j \cdot N/2^{k+1}}$

5: **for** $i = 0$ to $N/2^{k+1} - 1$ **do**

6: $a, b \leftarrow v_{i \cdot 2^k + j}, v_{i \cdot 2^k + 2^{k-1} + j}$

7: $v_{i \cdot 2^k + j} = a + u \cdot b$

8: $v_{i \cdot 2^k + 2^{k-1} + j} = a - u \cdot b$

9: **return** $(v_i)_{0 \leq i < N/2}$

D.5 Definition of the SlotToCoeff function

For convenience, we define the SlotToCoeff_N operation over the cyclotomic ring $\mathcal{S}_N$, but the procedure will indeed be the same over the integer ring $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$. The SlotToCoeff_N function takes as input a polynomial $v(X)$ encoding $N/2$ slots $\mathbf{t} = (t_0, \dots, t_{N/2-1}) \in \mathbb{R}^{N/2}$, that is $v(X) = \text{iDFT}_N(\mathbf{t})$. It outputs a polynomial $m(X) \in \mathbb{R}[X^2]/(X^N + 1)$, whose $N/2$ coefficients are the t_j 's:

$$m(X) = \text{SlotToCoeff}_N(v) = \sum_{j=0}^{N/2-1} t_j \cdot X^{2j}.$$

We can therefore write $m(X) = \tilde{m}(X^2)$ for $\tilde{m}(Y) = \sum_{j=0}^{N/2-1} t_j \cdot Y^j$, and we let $\boldsymbol{\alpha} = \text{DFT}_{N/2}(\tilde{m}) \in \mathbb{C}^{N/4}$. This implies $\text{DFT}_N(m) = (\boldsymbol{\alpha}, \boldsymbol{\alpha}) \in \mathbb{C}^{N/2}$.

We generalize the DFT function to take as input the coefficient vector of a polynomial, instead of the polynomial itself, with the same output. Therefore, since $\boldsymbol{\alpha} = \text{DFT}_{N/2}(\tilde{m})$ and by definition $\tilde{m}(Y) = \sum_{j=0}^{n-1} t_j \cdot Y^j$, we can write $\boldsymbol{\alpha} = \text{DFT}_{N/2}(\mathbf{t})$. In total, we can summarize the above in a commutative diagram:

$$\begin{array}{ccc} \text{Coefficient space} & & \text{Slot space} \\ v(X) & \xrightarrow{\text{DFT}_N} & \mathbf{t} \\ \text{SlotToCoeff} \downarrow & & \downarrow \text{DFT}_{N/2} \\ m(X) & \xrightarrow{\text{DFT}_N} & (\boldsymbol{\alpha}, \boldsymbol{\alpha}) \end{array}$$

From the above commutative diagram, the goal is therefore to compute the function $\mathbb{R}^{N/2} \rightarrow \mathbb{C}^{N/2}$, $\mathbf{t} \rightarrow (\text{DFT}_{N/2}(\mathbf{t}), \text{DFT}_{N/2}(\mathbf{t}))$ over the slot space, homomorphically over the coefficient space.

D.6 The SlotToCoeff algorithm

In fact, we cannot compute $\boldsymbol{\alpha} = \text{DFT}_{N/2}(\mathbf{t})$ directly with a fast $\mathcal{O}(\log N)$ algorithm. Either the input or the output must be indexed in bit-reversed order. We will use the function $\text{DFT}_{N/2, \text{bo}' \rightarrow \text{no}}$ computed by Alg. 7, whose input is bit-reversed and whose output is in normal order, which gives

$\text{DFT}_{N/2, \text{bo}' \rightarrow \text{no}}(\mathbf{t}') = \text{DFT}_{N/2}(\mathbf{t})$ where $\mathbf{t}' = \text{Br}'(\mathbf{t})$. We therefore have the updated commutative diagram:

$$\begin{array}{ccc}
 & \text{Coefficient space} & \text{Slot space} \\
 v(X) & \xrightarrow{\text{DFT}_N} & \text{Br}'(\mathbf{t}) \\
 \text{SlotToCoeff} \downarrow & & \downarrow \text{DFT}'_N \\
 m(X) & \xrightarrow{\text{DFT}_N} & (\boldsymbol{\alpha}, \boldsymbol{\alpha})
 \end{array}$$

where we define the function $\text{DFT}'_N(\mathbf{t}) = (\text{DFT}_{N/2, \text{bo}' \rightarrow \text{no}}(\mathbf{t}), \text{DFT}_{N/2, \text{bo}' \rightarrow \text{no}}(\mathbf{t}))$. The goal is therefore to evaluate the function $\text{DFT}'_N(\mathbf{t}')$ homomorphically over the coefficient space.

Algorithm 8 Fast computation of DFT'_N

Input: A sequence $(p_i)_{0 \leq i < N/2} \in \mathbb{R}^{N/2}$, $\zeta = \exp(I\pi/N)$ and $N = 2^\ell$.

Output: A sequence $(v_i)_{0 \leq i < N/2} = \text{DFT}'(\mathbf{p})$

```

1:  $(v_j)_{0 \leq j < N/2} = (\mathbf{w}, \mathbf{w})$ , where  $\mathbf{w} = (p_j + I \cdot p_{j+N/4})_{0 \leq j < N/4}$ 
2: for  $k = 1$  to  $\ell - 2$  do
3:   for  $j = 0$  to  $2^{k-1} - 1$  do
4:      $u = \zeta^{5^j \cdot N/2^{k+1}}$ 
5:     for  $i = 0$  to  $N/2^{k+1} - 1$  do
6:        $a, b \leftarrow v_{i \cdot 2^k + j}, v_{i \cdot 2^k + 2^{k-1} + j}$ 
7:        $v_{i \cdot 2^k + j} = a + u \cdot b$ 
8:        $v_{i \cdot 2^k + 2^{k-1} + j} = a - u \cdot b$ 
9: return  $(v_i)_{0 \leq i < N/2}$ 

```

To perform the homomorphic evaluation of the previous algorithm, we first rewrite the operations in vector form. For a vector $\mathbf{v} = (v_i)_{0 \leq i < N/2}$, we write

$$\text{Rot}_r(\mathbf{r}) = (v_{r+j \bmod N/2})_{0 \leq j < N/2} = (v_r, \dots, v_{N/2-1}, v_0, \dots, v_{r-1})$$

the rotation of $\mathbf{v}$ by r positions to the left. For a vector $\mathbf{a}$, we denote by $\mathbf{a}^n$ the repetition n times of $\mathbf{a}$.

For the first line, we can write:

$$\mathbf{v} = \mathbf{p} \odot ((1)^{N/4}, (I)^{N/4}) + \text{Rot}_{N/4}(\mathbf{p}) \odot ((I)^{N/4}, (1)^{N/4})$$

We can rewrite the first butterfly equation as:

$$v_{i \cdot 2^k + j} \leftarrow v_{i \cdot 2^k + j} + \zeta^{5^j \cdot N/2^{k+1}} \cdot (\text{Rot}_{2^{k-1}}(\mathbf{v}))_{i \cdot 2^k + j}$$

Similarly, we can rewrite the second butterfly equation as:

$$v_{i \cdot 2^k + 2^{k-1} + j} \leftarrow (\text{Rot}_{-2^{k-1}}(\mathbf{v}))_{i \cdot 2^k + 2^{k-1} + j} - \zeta^{5^j \cdot N/2^{k+1}} \cdot v_{i \cdot 2^k + 2^{k-1} + j}$$

We can therefore write:

$$\mathbf{v} \leftarrow \mathbf{v} \odot \boldsymbol{\omega}_0 + \text{Rot}_{2^{k-1}}(\mathbf{v}) \odot \boldsymbol{\omega}_1 + \text{Rot}_{-2^{k-1}}(\mathbf{v}) \odot \boldsymbol{\omega}_2$$

where

$$\begin{aligned}\omega_0 &= \left((1)^{2^{k-1}}, \left(-\zeta^{5^j \cdot N/2^{k+1}} \right)_{0 \leq j < 2^{k-1}} \right)^{N/2^{k+1}} \\ \omega_1 &= \left(\left(\zeta^{5^j \cdot N/2^{k+1}} \right)_{0 \leq j < 2^{k-1}}, (0)^{2^{k-1}} \right)^{N/2^{k+1}} \\ \omega_2 &= \left((0)^{2^{k-1}}, (1)^{2^{k-1}} \right)^{N/2^{k+1}}\end{aligned}$$

Finally, we can now define the **SlotToCoeff** algorithm, which is a homomorphic evaluation of the previous algorithm (Alg. 8).

Algorithm 9 SlotToCoeff

Input: A polynomial $v = \text{iDFT}_N(\text{Br}'(t))$ for $t \in \mathbb{R}^{N/2}$, $\zeta = \exp(I\pi/N)$ and $N = 2^\ell$.

Output: A polynomial $m(X) = \text{SlotToCoeff}_N(v) \simeq \sum_{j=0}^{N/2-1} t_j \cdot X^{2j}$

```

1:  $v \leftarrow v \times \text{Ecd}((1)^{N/4}, (I)^{N/4}) + \Psi_{N/4}(v) \times \text{Ecd}((I)^{N/4}, (1)^{N/4})$ 
2: for  $k = 1$  to  $\ell - 2$  do
3:    $w_0 = \left( (1)^{2^{k-1}}, \left( -\zeta^{5^j \cdot N/2^{k+1}} \right)_{0 \leq j < 2^{k-1}} \right) \in \mathbb{C}^{2^k}$ 
4:    $W_0 = \text{Ecd}(w_0)$ 
5:    $w_1 = \left( \left( \zeta^{5^j \cdot N/2^{k+1}} \right)_{0 \leq j < 2^{k-1}}, (0)^{2^{k-1}} \right) \in \mathbb{C}^{2^k}$ 
6:    $W_1 = \text{Ecd}(w_1)$ 
7:    $w_2 = \left( (0)^{2^{k-1}}, (1)^{2^{k-1}} \right) \in \mathbb{C}^{2^k}$ 
8:    $W_2 = \text{Ecd}(w_2)$ 
9:    $v \leftarrow v \times W_0 + \Psi_{2^{k-1}}(v) \times W_1 + \Psi_{-2^{k-1}}(v) \times W_2$ 
10: return  $v$ 
```

The above **SlotToCoeff** algorithm is essentially the same as in [CHH18], although in the latter it is described via a matrix approach. It has multiplicative depth $\log_2(N/2)$, and requires $2 \log_2(N/2) - 1$ homomorphic rotations and $3 \log_2(N/2) - 1$ external multiplications. Note that the polynomials W_0 , W_1 , and W_2 are independent of the input and can thus be precomputed.

D.7 SlotToCoeff for n slots

We have the updated commutative diagram:

$$\begin{array}{ccccc} \text{Coefficient space} & & \text{Coefficient space} & & \text{Slot space} \\ \mathbb{Z}[X^{N/(2n)}]/(X^N + 1) & & \mathbb{Z}[Y]/(Y^{2n} + 1) & & \mathbb{C}^n \\ \\ v(X) & \xrightarrow{\quad} & \tilde{v}(Y) & \xrightarrow{\text{DFT}_{2n}} & \text{Br}'(t) \\ \text{SlotToCoeff}_{N,n} \downarrow & & \text{SlotToCoeff}_{2n} \downarrow & & \downarrow \text{DFT}'_{2n} \\ m(X) & \xrightarrow{\quad} & \tilde{m}(Y) & \xrightarrow{\text{DFT}_{2n}} & (\alpha, \alpha) \end{array}$$

Algorithm 10 SlotToCoeff_{N,n}

Input: A polynomial $v = \text{iDFT}_N(\text{Br}'(\mathbf{t})^{N/(2n)})$ for $\mathbf{t} \in \mathbb{R}^n$, $\zeta = \exp(I\pi/(2n))$ and $n = 2^\ell$.

Output: A polynomial $m(X) = \text{SlotToCoeff}_{N,n}(v) \simeq \sum_{j=0}^{n-1} t_j \cdot X^{j \cdot N/n}$

```
1:  $v \leftarrow v \times \text{Ecd}((1)^{n/2}, (I)^{n/2}) + \Psi_{n/2}(v) \times \text{Ecd}((I)^{n/2}, (1)^{n/2})$ 
2: for  $k = 1$  to  $\ell - 1$  do
3:    $\mathbf{w}_0 = \left( (1)^{2^{k-1}}, (-\zeta^{5^j \cdot n/2^k})_{0 \leq j < 2^{k-1}} \right) \in \mathbb{C}^{2^k}$ 
4:    $W_0 = \text{Ecd}(\mathbf{w}_0)$ 
5:    $\mathbf{w}_1 = \left( (\zeta^{5^j \cdot n/2^k})_{0 \leq j < 2^{k-1}}, (0)^{2^{k-1}} \right) \in \mathbb{C}^{2^k}$ 
6:    $W_1 = \text{Ecd}(\mathbf{w}_1)$ 
7:    $\mathbf{w}_2 = \left( (0)^{2^{k-1}}, (1)^{2^{k-1}} \right) \in \mathbb{C}^{2^k}$ 
8:    $W_2 = \text{Ecd}(\mathbf{w}_2)$ 
9:    $v \leftarrow v \times W_0 + \Psi_{2^{k-1}}(v) \times W_1 + \Psi_{-2^{k-1}}(v) \times W_2$ 
10: return  $v$ 
```

D.8 Radix implementation

As in [CHH18], one can reduce the multiplicative depth by using radices. For a power-of-two radix r , [CHH18] shows that the homomorphic linear transform requires roughly $2r \log_r n$ homomorphic rotations and $3r \log_r n$ external multiplications. The advantage of the radix-based approach is that it yields a smaller multiplicative depth of around $\log_r n$ compared to the initial depth of $\log_2 n$. However, since the complexity increases as the radix does, the radix variant can be seen as a trade-off, which becomes disadvantageous for a too large r . As also described in [CHH18], without affecting the depth, the number of homomorphic rotations can be even further reduced to $O(\sqrt{r} \log_r n)$ by using a baby-step giant-step approach.

In practice, the authors of [CHH18] suggest to use $r = 2^5$ for a full slots implementation of CKKS, i.e. with common parameters $n = N/2$ and $N = 2^{15}$ or $N = 2^{16}$. They show that the running times for radices ranging from 2^1 to 2^4 are quite similar. However, for few slots n , we can use a smaller radix since a radix r is only of avail if $r \leq \sqrt{n}$.

E Proof of Theorem 1

We start with fresh encryptions of the secret-key bits s_k . By Lemma 1, their initial noise size is bounded by $E = 3N\kappa$. By Lemma 4, since we perform external products with encodings of real values of absolute value less than $\nu = 1$, the size of the noise of each product is at most $2\nu E + 8N \leq 14N\kappa$.

For the product tree, we must consider the error obtained when computing the product of the encryptions of two complex values, whose real and imaginary part are encrypted separately. Given the plaintexts $z_1 = x_1 + i \cdot y_1$ and $z_2 = x_2 + i \cdot y_2$, we have $z_1 z_2 = x_1 x_2 - y_1 y_2 + i \cdot (x_1 y_2 + x_2 y_1)$. Therefore, for $x = x_1 x_2 - y_1 y_2$, we must compute the homomorphic difference:

$$\text{ct}_x \leftarrow \text{MultR}_{\text{evk}}(\text{ct}_{x_1}, \text{ct}_{x_2}) - \text{MultR}_{\text{evk}}(\text{ct}_{y_1}, \text{ct}_{y_2}),$$

and similarly for ct_y for $y = x_1 y_2 + x_2 y_1$. Since all plaintext values x_1, y_1, x_2, y_2 are bounded by 1 in absolute value, the errors in $\text{MultR}(\text{ct}_{x_1}, \text{ct}_{x_2})$ and $\text{MultR}(\text{ct}_{y_1}, \text{ct}_{y_2})$ are upper bounded by $2E + 8N$, where E is an upper bound of the errors in ct_{x_1} , ct_{x_2} , ct_{y_1} and ct_{y_2} . Therefore, the error magnitudes of ct_x and ct_y are both upper-bounded by $4E + 16N$.

We now consider the product tree of depth $\ell = \log_2 N$. For $0 \leq i \leq \ell$, we can upper-bound the noise of the current ciphertext at level i by $u_i N \kappa$, where $u_0 = 14$, and the recursive relation $u_{i+1} = 4u_i + 16$ holds. One can show that $u_i \leq 20 \cdot 2^{2i} - 6$ by a recursive approach. For $i = \ell$, the error size after all multiplications is thus upper-bounded by $20 \cdot 2^{2\ell} N \kappa = 20N^3 \kappa$. We must therefore assume that $\Delta \geq (20N^3 \kappa)^2$ since we require $\Delta \geq E^2$ in each multiplication.

After the final scaling by $q/(2\pi\Delta)$, and by applying Lemma 4 again, the noise size becomes $20N^3 \kappa \cdot q/(2\pi\Delta) + 8N$. Therefore, for $\Delta \geq 4N^3 \kappa q$, the size of the noise is at most $9N$. This implies that by writing $m = \langle \text{ct}, \text{sk} \rangle(0) = \langle \mathbf{s}, \mathbf{c} \rangle \in \mathbb{Z}$, our final ciphertext ct' satisfies:

$$\langle \text{ct}', \text{sk} \rangle = \text{Ecd}(F(m)) + e' \pmod{p \cdot q},$$

for some e' with $\|e'\|_\infty \leq 9N$ and

$$F(m) = \frac{q}{2\pi\Delta} \sin\left(\frac{2i\pi m}{q}\right).$$

Given the condition $|m| < q^{2/3}$, we obtain by bounding the residual term in the Taylor expansion:

$$\left|F(m) - \frac{m}{\Delta}\right| \leq \frac{q}{2\pi\Delta} \cdot \frac{1}{3!} \cdot \left(\frac{2\pi|m|}{q}\right)^3 \leq \frac{7|m|^3}{q^2\Delta} \leq \frac{7}{\Delta}.$$

This implies $|\text{Ecd}(F(m)) - m| \leq \Delta \cdot |F(m) - m/\Delta| + 1 \leq 8$. We can eventually write:

$$\langle \text{ct}', \text{sk} \rangle = \langle \text{ct}, \text{sk} \rangle(0) + e_{\text{bt}} \pmod{p \cdot q},$$

where $\|e_{\text{bt}}\|_\infty \leq 10N$ as claimed.

F Algorithms for the single slot bootstrapping

Algorithm 11 Bootstrapping key generation, single slot

Input: A length N secret key $\mathbf{s}$ with Hamming weight h and $B = N/h$

Output: A bootstrapping key $\mathbf{cs} = (\mathbf{cs}_0, \mathbf{cs}_1)$

- 1: **for all** $0 \leq j < B/2$ **and** $0 \leq b < h$ **do**
 - 2: $\tilde{s}_{j \cdot h + b}^0 = s_{b \cdot B + j}$
 - 3: $\tilde{s}_{j \cdot h + b}^1 = s_{b \cdot B + B/2 + j}$
 - 4: $S_0(X), S_1(X) \leftarrow \text{Ecd}((\tilde{s}_i^0)_{0 \leq i < N/2}), \text{Ecd}((\tilde{s}_i^1)_{0 \leq i < N/2})$
 - 5: **return** $\mathbf{cs} = (\text{Enc}_{\text{pk}}(S_0(X)), \text{Enc}_{\text{pk}}(S_1(X)))$
-

Algorithm 12 Bootstrapping, single slot

Input: A modulus q , a bootstrapping key $\mathbf{cs}$, an RLWE ciphertext $\text{ct} = (b, a)$, and $\delta = (q/(4\pi\Delta))^{1/h}$

Output: A refreshed ciphertext ct'

- 1: $\mathbf{c} = (a_0 + b_0, -a_{N-1}, -a_{N-2}, \dots, -a_1)$
 - 2: **for all** $0 \leq j < B/2$ **and** $0 \leq b < h$ **do**
 - 3: $e_{j \cdot h + b}^0 = \exp(2i\pi \cdot c_{b \cdot B + j}/q) \cdot \delta$
 - 4: $e_{j \cdot h + b}^1 = \exp(2i\pi \cdot c_{b \cdot B + B/2 + j}/q) \cdot \delta$
 - 5: $E_0, E_1 \leftarrow \text{Ecd}((e_i^0)_{0 \leq i < N/2}), \text{Ecd}((e_i^1)_{0 \leq i < N/2})$
 - 6: $T_0, T_1 \leftarrow \text{ExtMultR}(\mathbf{cs}_0, E_0), \text{ExtMultR}(\mathbf{cs}_1, E_1)$
 - 7: **return** $\text{ct}' = \text{Im2}(\text{Pr}_{h \rightarrow 1}(\text{Tr}_{N/2 \rightarrow h}(T_0 + T_1)))$
-

G Bootstrapping algorithm for more than $B/2$ slots

The bootstrapping algorithm from Section 5.2 is limited to bootstrapping up to $n \leq n_{\max} = B/2 = N/(2h)$ components. However, it can be extended to support more slots, specifically $n' \leq N$, by shifting the coefficients of the input ciphertext and applying the previous bootstrapping procedure as a black box for each group of $n_{\max}$ coefficients.

More precisely, the previous bootstrapping algorithm (Alg. 2) inputs a ciphertext ct for the plaintext polynomial $m(X) = \sum_{i=0}^{N-1} m_i \cdot X^i$, and outputs a refreshed ciphertext for the polynomial $m'(X) = \sum_{i=0}^{n-1} m_{i \cdot N/n} \cdot X^{i \cdot N/n}$. In other words, only the n coefficients m_i of $m(X)$ such that $i \equiv 0 \pmod{N/n}$ are kept and bootstrapped, while the others are lost, because of the initial $C \leftarrow \text{DecMat}(\text{ct})$ procedure which only considers the coefficients of $X^{i \cdot N/n}$ in $m(X)$.

To bootstrap $n' > n$ coefficients, we can therefore proceed by repeatedly shifting the coefficients of the input plaintext (homomorphically on the input ciphertext), by a shifting polynomial $X^{-j \cdot N/n'}$. We obtain a bootstrapped ciphertext, which we shift back by $X^{j \cdot N/n'}$. We then compute the sum for all $0 \leq j < n'/n$ of all such ciphertexts. At the end, we obtain a refreshed ciphertext for all the n' components. We describe the concrete algorithm below. The number of homomorphic operations becomes $\mathcal{O}(n' + (n'/n) \cdot \log N)$, where $n = n_{\max} = B/2$ and $n' \leq N/2$, which gives $n'/n \leq h$. Assuming $h = \mathcal{O}(1)$, the complexity remains $\mathcal{O}(n' + \log N)$, while the depth remains unchanged.

Algorithm 13 Bootstrapping, more than $B/2$ slots

Input: A modulus q , a bootstrapping key $(\text{CS}_u)_{0 \leq u < 2n}$, an RLWE ciphertext $\text{ct} = (b, a)$ containing $n' \geq n$ slots

Output: A refreshed ciphertext ct'

```

1:  $\text{acc} \leftarrow (0, 0)$ 
2: for  $j = 0$  to  $n'/n - 1$  do
3:    $\text{ct}_s \leftarrow \text{ExtMult}(\text{ct}, X^{-j \cdot N/n'})$ 
4:    $\text{ct}'_s \leftarrow \text{Bootstrap}(q, \text{cs}, \text{ct}_s)$ 
5:    $\text{acc} \leftarrow \text{Add}(\text{acc}, \text{ExtMult}(\text{ct}'_s, X^{j \cdot N/n'}))$ 
6: return  $\text{acc}$ 

```
